
Development of a Security Mechanism for Blocking Malicious HTTP Requests on Endpoint Systems

By

ARTUR MERKAYEV
ABDIAKIM ABDIQADYR



School of Cybersecurity
ASTANA IT UNIVERSITY

6B06301 - Cybersecurity
Supervisor: Aliya Abdraman

ASTANA, JUNE 2026

Abstract

This bachelor thesis presents the design and implementation of an endpoint-level security mechanism for blocking malicious HTTP requests on a Linux endpoint. The proposed prototype intercepts incoming plaintext HTTP traffic before it reaches the local **Apache HTTP Server**. Traffic interception is performed through **iptables** and **NFQUEUE**, while request inspection is implemented in a **Python HTTP Guard Agent**.

The mechanism analyzes the **HTTP request line**, applies repeated URL decoding and normalization, and uses **rule-based detection** to identify indicators of **path traversal**, **XSS-like payload**, and **SQL injection-like payload**. Depending on the result, each request receives an **ACCEPT** or **DROP** decision. Every processed event is stored in a structured **CSV** log containing source and destination information, raw and normalized request lines, detected category, matched pattern, and action.

For monitoring and validation, a local **Agent API** and a **SIEM-like dashboard** were developed. The dashboard displays final experiment metrics, runtime event logs, validation results, event investigation details, normalization evidence, and evidence export functions. Experimental evaluation was conducted on a controlled dataset of 50 HTTP test cases. The prototype achieved 50 passed tests out of 50, a 100% pass rate, 20 accepted benign requests, 30 blocked suspicious requests, 0 false positives, and 0 false negatives. The mechanism was also verified in a scenario where a **Windows attacker client** sent requests to a protected **Linux endpoint**.

The results confirm that the proposed prototype demonstrates a reproducible and explainable endpoint-level approach to blocking malicious HTTP requests in a controlled laboratory environment.

Keywords: HTTP security, endpoint-level protection, **iptables**, **NFQUEUE**, rule-based detection, **SIEM-like dashboard**, malicious request blocking.

Dedication and acknowledgements

The authors express sincere gratitude to their supervisor, academic staff, and colleagues for their guidance, feedback, and support during the preparation of this thesis.

Authors' declaration

We declare that the work in this diploma work was carried out in accordance with the requirements of the University's Regulations and that it has not been submitted for any other academic award. Except where indicated by specific references in the text, the work is the candidates' own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the authors.

SIGNED: DATE:

Definitions

The following terms are used in this work:

HTTP request	A message sent by a client to a web server using the Hypertext Transfer Protocol. In this work, the main analyzed part of the request is the <code>request line</code> .
request line	The first line of an HTTP request containing the method, request target, and protocol version.
endpoint	A host system that receives network traffic and runs the protected service. In this thesis, the protected endpoint is a Linux machine running Apache HTTP Server.
endpoint-level protection	A protection approach in which traffic is analyzed and blocked directly on the host where the protected service is running.
rule-based detection	A detection approach based on matching request content against predefined patterns associated with suspicious behavior.
ACCEPT	A decision allowing the packet to continue normal processing and reach the protected service.
DROP	A decision discarding the packet before it reaches the protected service.

Designations and Abbreviations

The following designations and abbreviations are used in this work:

API	Application Programming Interface.
CSV	Comma-Separated Values.
HTTP	Hypertext Transfer Protocol.
HTTPS	Hypertext Transfer Protocol Secure.
IDS/IPS	Intrusion Detection System / Intrusion Prevention System.
NFQUEUE	Netfilter queue mechanism for passing packets from the Linux kernel to user space.
SIEM	Security Information and Event Management.
SQLI	SQL injection-like request category used in the prototype.
TCP	Transmission Control Protocol.
TRAVERSAL	Path traversal-like request category used in the prototype.
URI	Uniform Resource Identifier.
WAF	Web Application Firewall.
XSS	Cross-Site Scripting-like request category used in the prototype.

List of Tables

TABLE	Page
1.1 Comparison of approaches to HTTP traffic filtering	5
2.1 Main categories of rule-based detection	14
2.2 Event structure in the HTTP Guard CSV log	15
3.1 Components of the laboratory environment	19
3.2 Main local Agent API endpoints	25
3.3 Sections of the SIEM-like dashboard	26
3.4 Categories of test HTTP requests	28
3.5 Result files of the controlled experiment	28
3.6 Final indicators of the controlled experiment	29
3.7 Directions for further prototype development	35
1 Main source code and configuration files	56
2 Main experiment result files	57
3 Key final experiment indicators	57

List of Figures

FIGURE	Page
3.1 Simplified architecture of the proposed HTTP Guard mechanism.	20
3.2 Core implementation fragment of the HTTP Guard Agent , including request-line normalization, rule-based detection, structured logging, and the final ACCEPT or DROP decision.	23
3.3 Input-chain iptables rule redirecting incoming TCP traffic on port 80 to NFQUEUE , allowing the HTTP Guard Agent to inspect requests before they reach Apache.	24
3.4 Dashboard view of the official controlled experiment metrics: 50 test cases, 100% pass rate, 20 accepted benign requests, 30 dropped malicious requests, and zero false positives or false negatives.	27
3.5 Full 50-test live validation result displayed in the dashboard, confirming that all prepared HTTP test cases produced the expected decisions.	30
3.6 Windows PowerShell commands used to send benign and suspicious HTTP requests to the protected Linux endpoint during the remote demonstration scenario.	31
3.7 HTTP Guard terminal output on the protected Linux endpoint, showing events generated by requests from the Windows attacker client and the corresponding ACCEPT or DROP actions.	32
3.8 Dashboard investigation panel for a SQL injection-like event, including the raw request line, normalized request line, matched pattern, category, action, and explanation.	33
1 Simplified architecture of the HTTP Guard mechanism.	43
2 Request processing algorithm of the HTTP Guard prototype.	44
3 Final project structure of the practical implementation.	46
4 Dashboard metrics for the final controlled experiment.	47

5	Full live validation result for the 50 HTTP test cases.	47
6	Windows client commands used to test the protected endpoint.	48
7	HTTP Guard event output for requests from the Windows client.	49
8	SIEM-like Event Explorer showing remote-client events.	50
9	Investigation panel for an SQLi-like request event.	50
10	iptables rule redirecting incoming HTTP traffic to NFQUEUE.	51
11	Agent API terminal output during dashboard support.	51
12	Dashboard server terminal output during local monitoring.	51
13	Quick live self-test result showing successful completion of four readiness checks.	52
14	Runtime HTTP Guard log overview showing accumulated event statistics.	52
15	Normalization evidence for an encoded malicious request.	53
16	Dashboard controls for exporting filtered evidence.	53
17	Technical diagnostics panel for required project files.	53
18	Generated live validation result files.	54
19	Tail of the structured HTTP Guard CSV event log.	54
20	Core HTTP Guard logic for normalization, detection, logging, and packet decisions.	55

Table of Contents

Abstract	i
Dedication and acknowledgements	ii
Authors' declaration	iii
Definitions	iv
Designations and Abbreviations	v
List of Tables	vi
List of Figures	vii
 Introduction	 xii
 1 Theoretical Foundations of Protecting Endpoint Systems from Malicious HTTP Requests	 1
1.1 HTTP Request as an Object of Analysis	1
1.2 Malicious HTTP Requests and Main Attack Classes	2
1.2.1 Path Traversal	3
1.2.2 Cross-Site Scripting-like payload	3
1.2.3 SQL Injection-like payload	4
1.3 Approaches to Filtering and Blocking HTTP Traffic	4
1.4 Protection at the Endpoint Level	5
1.5 Normalization as Part of Malicious Request Detection	6
1.6 Rule-based detection: Advantages and Limitations	7
1.7 Event Monitoring and the Role of the SIEM-like dashboard	8

1.8	Research Gap and Rationale for the Selected Approach	9
2	Design of the HTTP Request Blocking Mechanism	10
2.1	Requirements for the Developed Mechanism	10
2.2	General Solution Architecture	12
2.3	HTTP Request Processing Model	13
2.4	Detection Rules and Request Line Normalization	14
2.5	Logging and Monitoring Structure	15
2.6	Laboratory Usage Scenarios	16
2.7	Limitations of the Designed Solution	16
3	Prototype Implementation and Experimental Evaluation	18
3.1	Development and Testing Environment	18
3.2	Implementation of the HTTP Guard Agent	21
3.3	Configuration of iptables and NFQUEUE	24
3.4	Implementation of the Agent API	25
3.5	SIEM-like Dashboard	26
3.6	Data Collection Methodology and Test Dataset	28
3.7	Controlled Experiment on 50 HTTP Test Cases	29
3.8	Verification of the Windows Attacker Client to Linux Protected Endpoint Scenario	30
3.9	Analysis of Results	33
3.10	Limitations and Directions for Further Development	34
	Conclusion	37
	Bibliography	39
	Appendix A	42
	Architectural Diagrams	42
	General System Architecture	42
	Request Processing Algorithm Flowchart	43
	Appendix B	45
	Screenshots of the Practical Implementation	45
	Final Project Structure	45
	Controlled Experiment Metrics	46

Full 50-Test Live Validation	47
Windows Attacker Client	47
HTTP Guard Events from the Windows Client	48
SIEM-like Event Explorer	49
Event Investigation Panel	50
iptables Rule	50
Agent API and Dashboard Server	51
Quick Self-Test and Runtime Overview	51
Normalization Evidence	52
Export Evidence and Diagnostics	53
Result files and structured CSV log	53
Main HTTP Guard Logic	54
Appendix C	56
Source Code Fragments	56
Appendix D	57
Experiment Results	57

Introduction

The relevance of this thesis is determined by the fact that HTTP remains one of the main protocols used for communication between clients and web services. Through HTTP requests, users pass parameters, access resources, submit forms, and interact with application programming interfaces. At the same time, an HTTP request can also be used by an attacker to deliver a malicious payload aimed at exploiting weaknesses in a web application or server.

Common indicators of malicious HTTP requests include attempts at **path traversal**, **XSS-like payload** delivery, and **SQL injection-like payload** delivery. Such requests may contain attempts to access system files, inject script code, submit **SQL-like** expressions, or hide a payload through encoded representations. Even when a specific application is not vulnerable, the presence of such patterns in traffic is an important indicator of suspicious activity and should be recorded by security mechanisms.

Traditional approaches to protecting web services include **application-level validation**, **Web Application Firewall** solutions, **IDS/IPS** technologies, and centralized event monitoring. This work focuses on a narrower applied approach: protection at the endpoint level. In this approach, an HTTP request is analyzed directly on a **Linux endpoint** before it is passed to the local web service. This makes it possible to block a suspicious request before it is processed by **Apache HTTP Server**.

The research problem is the need to design a simple, reproducible, and explainable mechanism that can intercept HTTP requests on an endpoint system, analyze their indicators, make a blocking decision, and store the results for further monitoring. The decision must be explainable: for each event, it should be possible to identify the received request line, the matched pattern, the assigned category, and the action taken.

The aim of the thesis is to develop a security mechanism for blocking malicious HTTP requests on endpoint systems and to evaluate its operation experimentally in a controlled laboratory environment.

To achieve this aim, the following tasks were defined:

1. to study the main types of malicious HTTP requests and approaches to their detection;
2. to define requirements for an `endpoint-level` HTTP request blocking mechanism;
3. to design the architecture of a solution based on the `Linux packet filtering layer`, `NFQUEUE`, and `user-space inspection`;
4. to implement a `Python HTTP Guard Agent` for analyzing the `request line` and making `ACCEPT/DROP` decisions;
5. to implement `request line` normalization for detecting encoded payload variants;
6. to prepare `rule-based detection` for the `TRAVERSAL`, `XSS`, and `SQLI` categories;
7. to organize structured event logging;
8. to develop a local `Agent API` and a `SIEM-like dashboard` for monitoring the results;
9. to prepare a dataset of 50 HTTP test cases and conduct a controlled experiment;
10. to verify the prototype in the `Windows Attacker Client to Linux Protected Endpoint` scenario;
11. to analyze the results, limitations, and directions for further development.

The object of the research is the processing of HTTP requests on an endpoint system.

The subject of the research is a mechanism for intercepting, analyzing, and blocking malicious HTTP requests on a `Linux endpoint`.

The research methods include analysis of existing approaches to HTTP traffic filtering, architectural design of `endpoint-level protection`, prototype implementation in `Python`, configuration of `iptables` and `NFQUEUE`, experimental evaluation on a `controlled dataset`, analysis of `structured logs`, and visualization of results through a `SIEM-like dashboard`.

The scientific and practical significance of the work lies in the development of a reproducible prototype that demonstrates the complete cycle of `endpoint-level protection`: interception of an HTTP request, normalization of the `request line`, `rule-based detection`, an `ACCEPT` or `DROP` decision, event logging, and visualization of the result in a `dashboard`. The practical part can be used as an educational and laboratory example

for demonstrating the principles of `endpoint traffic inspection` and security event monitoring.

The first chapter discusses the theoretical foundations of protecting endpoint systems from malicious HTTP requests, including the main attack classes, filtering approaches, and the role of normalization. The second chapter presents the design of the mechanism, including requirements, architecture, request processing model, and monitoring structure. The third chapter describes the practical implementation of the prototype, experimental evaluation, results of the `controlled experiment`, and verification with an external `Windows attacker client`.

Chapter 1

Theoretical Foundations of Protecting Endpoint Systems from Malicious HTTP Requests

This chapter presents the theoretical foundations required for developing a security mechanism aimed at blocking malicious HTTP requests on endpoint systems. The discussion focuses on the characteristics of HTTP traffic, common classes of web attacks, approaches to request filtering, and the rationale for the `endpoint-level` approach used in the practical part of the project.

In this thesis, a malicious HTTP request is understood as a request that contains indicators of an attempt to exploit a web application or web service through the `URI`, `query string`, or other elements of an HTTP message. The practical part focuses on three classes of indicators: `path traversal`, `cross-site scripting-like payload`, and `SQL injection-like payload`.

1.1 HTTP Request as an Object of Analysis

HTTP is an application-layer protocol used for data exchange between clients and servers [1, 2]. In a typical scenario, a client sends an HTTP request, and the server processes it and returns an HTTP response. A request may contain a method, a resource path, request parameters, headers, and, in some cases, a message body.

For this project, the first line of an HTTP request, the `request line`, was selected as

the main object of analysis. It contains the HTTP method, the `request target`, and the protocol version. An example of a `request line` is:

```
GET /search?q=test HTTP/1.1
```

The `request line` often contains data that can be used by an attacker to deliver a malicious `payload`. For example, the URI or `query string` may contain an attempt to access files outside the web server directory, inject JavaScript code, or submit an SQL-like expression.

Analyzing the `request line` has several advantages for a laboratory prototype:

1. the `request line` is present in most HTTP requests;
2. it can be extracted conveniently from `plaintext HTTP traffic`;
3. many demonstrative `attack payloads` are placed in the URI or `query string`;
4. the analysis result is easy to log and explain;
5. this approach makes it possible to focus on interception and blocking before the request reaches the web server.

At the same time, analysis of only the `request line` has limitations. It does not cover payloads transmitted in the HTTP `body`, complex `multi-packet` scenarios, or encrypted HTTPS traffic. These limitations are considered when interpreting the results and are treated as directions for further work.

1.2 Malicious HTTP Requests and Main Attack Classes

Web applications and HTTP services frequently receive user input through URL paths, `query parameters`, forms, and API requests. If these inputs are not validated or processed correctly, an attacker may attempt to submit a malicious `payload`. This work considers three classes of indicators: `path traversal`, XSS-like `payload`, and SQLi-like `payload`.

1.2.1 Path Traversal

`Path traversal` is an attack class in which an attacker attempts to access files or directories outside the permitted area of a web application [3–5]. A typical indicator of this attack is the use of sequences such as `../`, which allow navigation upward in the directory tree.

An example of a `path traversal`-like request is:

```
GET /download?file=../../etc/passwd HTTP/1.1
```

In Linux systems, the file `/etc/passwd` is frequently used in demonstration examples because it is a well-known system file. Even if a specific application does not return this file, the presence of such a string in a request is a suspicious indicator and may be used to classify the request as `TRAVERSAL`-like.

`Path traversal` may appear not only in a direct form but also through encoding. For example, the sequence `../` may be represented as `%2e%2e%2f`, while a double-encoded form may appear as `%252e%252e%252f`. Therefore, the design of a detection mechanism must take `request line` normalization into account.

1.2.2 Cross-Site Scripting-like payload

`Cross-Site Scripting`, or XSS, is associated with injecting script content into a web page or context that may later be processed by a user’s browser [6–8]. In a complete application scenario, XSS depends not only on the incoming request but also on how the server stores, reflects, or renders user-controlled data.

In this project, the term `XSS-like payload` is used. This means that the mechanism does not prove exploitation of XSS in an application; instead, it detects characteristic signs of script-like input in the `HTTP request line`.

An example of an `XSS`-like request is:

```
GET /search?q=<script>alert(1)</script> HTTP/1.1
```

Strings containing `javascript:`, event handlers such as `onerror=`, or other signs of attempted client-side script injection may also be treated as suspicious. For `endpoint-level` filtering, these indicators can be used as a basis for blocking a request before it is processed by the web service.

1.2.3 SQL Injection-like payload

`SQL Injection` is associated with attempts to alter the structure of an `SQL` query through user input [9, 10]. In a real web application, such an attack becomes possible when input is included directly in an `SQL` query without parameterization or proper handling.

This work uses the term `SQLi-like payload` because the prototype analyzes only the `HTTP request line` and does not connect to a database. Its task is to detect indicators commonly associated with `SQL injection attempts`: quotes, logical expressions, `SQL` comments, and constructions such as `UNION SELECT`.

An example of an `SQLi-like` request is:

```
GET /login?u=admin' OR 1=1-- HTTP/1.1
```

Such a request may be classified as `suspicious` even if the tested web service has no database. This corresponds to the goal of the diploma project: blocking `HTTP` requests with indicators of a malicious payload at the `endpoint` level.

1.3 Approaches to Filtering and Blocking HTTP Traffic

Several approaches can be used to filter `HTTP` traffic and protect web services. They differ in placement, depth of analysis, configuration complexity, and degree of integration with the application.

`Application-level validation` is performed inside the web application itself. This approach is essential for complete security because only the application fully understands its business logic, allowed parameter values, and data context [9, 10]. However, `application-level validation` requires changes to application code and depends on the quality of development.

Web Application Firewall solutions are placed in front of a web application and analyze HTTP traffic for known attack patterns. A WAF may operate as a **reverse proxy**, a web server module, or a cloud service. This approach is useful for centralized protection, but it may require careful rule configuration and ongoing maintenance.

Network-level filtering is performed at the level of network rules, a **firewall**, or **IDS/IPS**. It is effective for filtering by IP addresses, ports, protocols, and selected signatures, but it is not always convenient for analyzing application-level HTTP content.

Endpoint-level interception means that the mechanism is installed directly on the protected endpoint and analyzes traffic before it is passed to the local service. This is the approach used in the present work. Its advantage is that blocking is performed on the **endpoint** itself, and the decision can be connected with local context such as port, service, event log, and host configuration.

Approach	Placement	Advantages	Limitations
Application-level validation	Application code	Accounts for business logic and data context	Requires correct development and application changes
WAF	In front of the web application	Centralized HTTP filtering	Requires configuration, rule maintenance, and may be bypassed
Network-level filtering	Network perimeter or firewall	Convenient for filtering by IP, ports, and protocols	Limited for application-level HTTP content analysis
Endpoint-level interception	Protected host	Blocks requests before local service processing	Requires an agent and endpoint configuration

Table 1.1: Comparison of approaches to HTTP traffic filtering

1.4 Protection at the Endpoint Level

Endpoint-level protection means that the security mechanism operates directly on the protected host. In this work, the protected host is a **Linux** system receiving HTTP requests on port 80. Blocking is performed before the request is passed to **Apache HTTP Server**.

This approach has several advantages:

1. protection operates locally on the same system where the service is hosted;

2. blocking occurs before the request is processed by the web server;
3. local indicators can be logged, including `source IP`, `destination IP`, `port`, `request line`, and `action`;
4. the prototype can be deployed in a controlled laboratory environment;
5. the mechanism can be extended to multiple `Linux endpoint` systems.

However, the `endpoint-level` approach does not replace other security measures. It should be considered an additional layer of protection. The application must still validate input correctly, and the infrastructure must use secure configuration, updates, access control, and monitoring.

In the project, `endpoint-level protection` is implemented through `iptables`, `NFQUEUE`, and the `Python HTTP Guard Agent`. This makes it possible to redirect HTTP packets from the `Linux kernel` to a `user-space` program, where more flexible analysis logic can be applied than in a conventional `firewall` rule.

1.5 Normalization as Part of Malicious Request Detection

One practical problem in filtering HTTP requests is that the same logical `payload` may be represented in different forms. For example, `path traversal` may appear as `../`, `%2e%2e%2f`, or a `double-encoded` variant [3, 4]. An `XSS-like payload` may also use different case, encoded characters, or alternative notations.

If only the `raw request line` is analyzed, some suspicious requests may not match the rules. Therefore, before rule checking, it is useful to transform the string into a more unified form. In this work, normalization includes:

1. repeated URL decoding;
2. replacement of backslashes with forward slashes;
3. conversion to lowercase.

Normalization is not a universal solution. More complex bypass techniques may require extended **canonicalization**, **Unicode** handling, application-context analysis, and deeper **HTTP** parsing. Nevertheless, even basic normalization improves **rule-based detection** in the laboratory prototype compared with analyzing only the raw request.

1.6 Rule-based detection: Advantages and Limitations

Rule-based detection is based on comparing analyzed data with predefined patterns. In this project, the rules are grouped into the categories **TRAVERSAL**, **XSS**, and **SQLI**. If the **request line** contains one of the patterns, the request is classified as **suspicious** and blocked.

The advantages of the **rule-based** approach are:

1. simplicity of implementation;
2. explainability of the decision;
3. the ability to show the **matched pattern**;
4. suitability for laboratory evaluation;
5. a transparent relation between the rule, category, and action.

Explainability is particularly important for this diploma project. During defense, it is possible to show which pattern was detected, what the normalized string looked like, and why the **DROP** decision was made.

The limitations of the **rule-based** approach are:

1. dependence on the completeness of the rule set;
2. possible bypass through new payload forms;
3. risk of **false positives** when rules are too broad;
4. limited ability to detect unknown attacks;

5. the need for regular rule updates.

For these reasons, `rule-based detection` is treated in this work as an understandable and reproducible foundation for the prototype. More complex methods, including `ML-based classification`, are considered a potential direction for future development after a sufficiently large labeled `dataset` has been collected.

1.7 Event Monitoring and the Role of the SIEM-like dashboard

Blocking a suspicious request is only one part of protection. For incident analysis, it is also important to understand what happened: who sent the request, which `endpoint` received it, what request line was processed, which pattern matched, and what action was taken. Therefore, logging and monitoring are treated as important parts of the project.

The `SIEM-like dashboard` in this work is not a full-scale `SIEM` system. It does not correlate events from multiple sources and does not replace an industrial monitoring platform. Its purpose is to demonstrate key event analysis functions at the prototype level:

1. displaying final experiment metrics;
2. viewing `runtime` events;
3. filtering by `category` and `action`;
4. searching by `request line`, `pattern`, `source`, and `destination`;
5. providing an `investigation` panel for a selected event;
6. exporting `evidence` in `CSV/JSON`;
7. supporting `live validation` for checking protection readiness.

This `dashboard` strengthens the practical part of the work because it demonstrates not only the fact of blocking but also the possibility of analyzing the result. This is important in a `security operation context`, where explainability, reproducibility, and evidence are essential.

1.8 Research Gap and Rationale for the Selected Approach

Existing approaches to protecting HTTP services often focus either on `application-level validation` or on the use of WAF, IDS/IPS, or centralized monitoring systems. These solutions are important, but this diploma project adopts a narrower applied focus: developing a reproducible `endpoint-level prototype` that intercepts HTTP requests on a Linux host, analyzes the `request line`, and blocks `suspicious traffic` before it reaches Apache.

The research gap addressed in this work can be formulated as follows: a simple, explainable, and reproducible mechanism is needed to demonstrate the full cycle of `endpoint-level HTTP request blocking`, from packet interception to visualization of the decision in a `dashboard`. Such a mechanism should not depend on cloud infrastructure or an external WAF service and should be suitable for laboratory evaluation.

The selected approach addresses this task through the combination of several components:

1. `iptables` and `NFQUEUE` intercept HTTP traffic on a Linux `endpoint`;
2. the `Python HTTP Guard Agent` performs `normalization` and `rule-based detection`;
3. a `CSV log` records every decision;
4. the `Agent API` provides `read-only` access to data;
5. the `SIEM-like dashboard` visualizes results and supports `live validation`;
6. a `controlled dataset` of 50 `test cases` enables formal evaluation of prototype correctness.

Thus, the theoretical basis of the work connects three areas: web attacks delivered through HTTP requests, `endpoint-level traffic interception`, and security event monitoring. On this basis, the second chapter designs the mechanism architecture, and the third chapter demonstrates its practical implementation and experimental evaluation.

Chapter 2

Design of the HTTP Request Blocking Mechanism

This chapter presents the design of a security mechanism for blocking malicious HTTP requests on endpoint systems. While the third chapter focuses on implementation and experimental evaluation, this chapter describes the design logic: mechanism requirements, solution architecture, request processing model, detection rule structure, logging, monitoring, and limitations of the selected approach.

The developed mechanism is intended to protect a `Linux endpoint` that hosts an HTTP service. The main idea is to analyze an HTTP request before it is processed by the web server. To achieve this, incoming HTTP traffic is redirected to a `user-space inspection layer`, where the `Python HTTP Guard Agent` normalizes the `request line`, checks it against `rule-based` indicators, and makes an `ACCEPT` or `DROP` decision.

2.1 Requirements for the Developed Mechanism

At the design stage, functional and non-functional requirements were defined for the prototype. Functional requirements describe the actions that the mechanism must perform, while non-functional requirements describe the constraints and properties necessary for a reproducible laboratory implementation.

The functional requirements are:

1. the mechanism must intercept HTTP traffic on the protected `Linux endpoint`;

2. analysis must be performed before the request is passed to the web server;
3. the mechanism must extract the `HTTP request line` from the incoming packet;
4. the `request line` must be normalized to detect encoded payload variants;
5. the system must detect indicators of `path traversal`, `XSS-like`, and `SQLi-like` requests;
6. `benign` requests must receive the `ACCEPT` decision;
7. `suspicious` requests must receive the `DROP` decision;
8. every processed event must be written to a structured log;
9. results must be available for monitoring through a `dashboard`;
10. it must be possible to conduct a `controlled experiment` and `live validation`.

The non-functional requirements are:

1. the prototype must operate in a controlled laboratory environment without external cloud dependencies;
2. the implementation must be reproducible on a `Linux` system;
3. detection rules must be understandable and explainable;
4. the `dashboard` must not perform traffic blocking and must be used only for monitoring;
5. the project must store results in files suitable for further analysis;
6. the architecture must allow further extension, for example support for multiple `endpoint` systems or integration with a `SIEM`.

These requirements were selected in accordance with the goal of the diploma project. The main task is to demonstrate an `endpoint-level protection mechanism`, not to develop an industrial `WAF` platform. Therefore, the design prioritizes architectural clarity, experiment reproducibility, and explainability of decisions.

2.2 General Solution Architecture

The solution architecture consists of four main layers:

1. **Attacker / client side**: the source of **benign** or **suspicious** HTTP requests. The client may be a **Linux** or **Windows** host using **curl** or a browser.
2. **Protected Linux endpoint**: the endpoint system where **Apache HTTP Server** and the **HTTP Guard Agent** are deployed.
3. **Inspection and decision layer**: the combination of **iptables**, **NFQUEUE**, and the **Python HTTP Guard Agent**, which performs interception, analysis, and decision-making [11–14].
4. **Monitoring and results layer**: **CSV logs**, the **Agent API**, and the **SIEM-like dashboard**, which are used for event analysis and result visualization.

The main processing flow is:

```
HTTP request -> iptables INPUT rule -> NFQUEUE -> Python HTTP  
Guard Agent -> ACCEPT/DROP -> log -> dashboard
```

If a request is classified as **BENIGN**, the packet is allowed and passed to **Apache HTTP Service**. If the request contains indicators of a malicious **payload**, the packet is dropped before web server processing. Thus, blocking is performed on the protected **Linux endpoint**, while the **dashboard** is used only as a visualization and analysis layer.

The selected architecture can also be extended in a laboratory environment to multiple **endpoint** systems. In such a case, each protected **Linux endpoint** must have its own **HTTP Guard Agent** and local traffic interception configuration. The **dashboard** may be used for monitoring and verification, but the enforcement decision remains local to each **endpoint**.

2.3 HTTP Request Processing Model

The HTTP request processing model is centered on analysis of the first request line, the `HTTP request line`. In a typical HTTP request, it contains the method, path, and protocol version, for example:

```
GET /search?q=test HTTP/1.1
```

For this project, the `request line` was selected as the main object of analysis for several reasons. First, many `path traversal`, `XSS-like`, and `SQLi-like` indicators appear in the `URI` and `query string`. Second, this approach simplifies packet processing and allows the work to focus on interception and blocking. Third, the `request line` is convenient to log and display in the `dashboard`.

The general processing model includes the following stages:

1. receiving a packet from `NFQUEUE` [11];
2. checking that the packet contains TCP traffic destined for `destination port 80`;
3. extracting the `HTTP request line`;
4. checking the `HTTP method`;
5. normalizing the `request line`;
6. comparing the `raw` and `normalized request line` with detection rules;
7. assigning a request category;
8. making an `ACCEPT` or `DROP` decision;
9. writing the event to a log;
10. returning the decision to `NFQUEUE`.

This model makes the decision process explainable. For every event, it is possible to show the original request line, the normalized request line, the matched pattern, the category, and the action. This is important for the thesis defense because the commission can see not only the final `DROP` result but also the reason for the decision.

2.4 Detection Rules and Request Line Normalization

The detection mechanism is based on a **rule-based** approach. The project uses three main **suspicious** categories [3, 6, 9].

Category	Purpose	Example indicators
TRAVERSAL	Detection of path traversal	../, %2e%2e, attempts to access system files
XSS	Detection of XSS-like payload	<script>, javascript:, onerror=
SQLI	Detection of SQL injection-like payload	' OR 1=1, UNION SELECT, --

Table 2.1: Main categories of **rule-based** detection

If no match is found against suspicious rules, the request is classified as **BENIGN**.

Normalization of the **request line** is an important design element. Without normalization, some malicious requests may be hidden through **percent-encoding** or case variation. For example, **path traversal** may appear not as `../`, but as `%2e%2e%2f`, or as the **double-encoded** variant `%252e%252e%252f`. Therefore, before rule checking, repeated URL decoding is performed, backslashes are replaced with forward slashes, and the string is converted to lowercase.

An example of normalization is:

```
Raw request line: GET /download?file=%252e%252e%252fetc%252fpasswd
HTTP/1.1
```

```
Normalized request line: get /download?file=../etc/passwd
http/1.1
```

Normalization is not complete protection against all bypass techniques, but it increases the robustness of **rule-based** checking compared with analysis of only the **raw request line**. As a result, the same malicious request meaning can be detected even when the **payload** is represented in different forms.

2.5 Logging and Monitoring Structure

Every processed event is written to the CSV log `logs/http_guard_events.csv`. The following fields are recorded for analysis and visualization.

Field	Description
<code>timestamp</code>	Time when the event was processed
<code>src_ip</code>	Source IP address of the request
<code>src_port</code>	Source port
<code>dst_ip</code>	IP address of the protected endpoint
<code>dst_port</code>	Destination port
<code>raw_request_line</code>	Original HTTP request line
<code>normalized_request_line</code>	Normalized HTTP request line
<code>category</code>	Request category
<code>matched_pattern</code>	Matched rule-based pattern
<code>action</code>	Decision: ACCEPT or DROP

Table 2.2: Event structure in the HTTP Guard CSV log

This log format was selected because it is simple and reproducible. A CSV file can be inspected manually, processed with Python scripts, imported into spreadsheets, or displayed in the dashboard. In addition, the CSV log separates the blocking mechanism from the visualization layer: HTTP Guard records events, and the dashboard reads the prepared data.

For monitoring, a SIEM-like dashboard was designed. It performs several functions:

1. displays final metrics of the controlled experiment;
2. shows a runtime log overview;
3. provides an event explorer with filters, search, and pagination;
4. shows an investigation panel for a selected event;
5. supports quick self-test and full 50-test live validation;
6. provides export evidence in CSV/JSON.

The dashboard is not part of the enforcement layer. It does not block traffic and does not modify agent decisions. Its role is analysis, visualization, and confirmation of results.

2.6 Laboratory Usage Scenarios

Two main laboratory usage scenarios were identified during the design stage.

The first scenario is the `attacker client` scenario. In this case, a `Windows` or `Linux` machine acts only as a client that sends HTTP requests to the protected `Linux endpoint`. The `attacker client` does not require installation of `HTTP Guard`; it is sufficient to use `curl` or a browser. This scenario was used to verify the `Windows Attacker Client to Linux Protected Endpoint` scenario.

The second scenario is the `protected endpoint` scenario. In this case, the `Linux` machine itself is the protected `endpoint` and must have `Apache`, an `iptables` rule, `NFQUEUE`, the `Python HTTP Guard Agent`, and the `Agent API`. If a second `Linux endpoint` must be protected, for example a teammate's machine, it must also run its own `HTTP Guard Agent`. Blocking is local to each protected system.

Thus, the `endpoint name` in the configuration is only a label for monitoring convenience. Protection does not work “by name”. It depends on the network address, local filtering rule, and running agent on the specific `Linux` host.

2.7 Limitations of the Designed Solution

The boundaries of the prototype were defined in advance during the design stage. This made it possible to avoid overstated expectations and to interpret the experimental results correctly.

The main limitations are:

1. only `plaintext` HTTP traffic is analyzed;
2. the main object of analysis is the `request line`;
3. `detection` is based on the `rule-based` approach;
4. the rule set is limited to the selected attack classes;
5. evaluation is performed in a controlled laboratory environment;
6. the `dashboard` is used only for monitoring and `validation`, not for `enforcement`;

7. **multi-endpoint monitoring** is treated as a possible extension rather than a required part of the current prototype.

These limitations do not reduce the value of the prototype for the diploma project, because the goal is to develop and experimentally validate a mechanism for blocking malicious HTTP requests on an **endpoint** system. Within this goal, the selected architecture demonstrates the complete cycle: packet interception, **request line** analysis, decision-making, logging, result visualization, and verification through an external **client scenario**.

Chapter 3

Prototype Implementation and Experimental Evaluation

This chapter presents the practical implementation of the security mechanism for blocking malicious HTTP requests on an endpoint system. The main emphasis is placed on verifying the prototype in a controlled laboratory environment where the protected node is a **Linux endpoint** running a local **Apache HTTP** service. Request analysis and decision-making are performed before the HTTP request is passed to the web server.

The practical part of the project consists of several connected components: the **HTTP Guard Agent**, traffic interception through **iptables** and **NFQUEUE**, detection rules, structured logging, a local **Agent API**, and a **SIEM-like dashboard** for event monitoring. In addition, a **controlled experiment** with 50 HTTP test cases, full live validation, and a scenario with an external **Windows attacker client** sending requests to a protected **Linux endpoint** were conducted.

The purpose of the practical implementation was not to create an industrial **WAF** solution, but to demonstrate an **endpoint-level protection mechanism**. In this approach, an HTTP request is intercepted on the **Linux host**, normalized, checked against **rule-based** indicators, and then either allowed or blocked.

3.1 Development and Testing Environment

The prototype was developed and tested in a laboratory environment based on **Linux**. The protected endpoint was a **Linux endpoint** running **Apache HTTP Server** on port

80. This configuration made it possible to reproduce a typical scenario in which an endpoint receives HTTP requests, while the traffic is additionally analyzed before web server processing.

Incoming HTTP packets were intercepted using `iptables` and `NFQUEUE`. The `iptables` rule redirected TCP traffic arriving on port 80 to an `NFQUEUE` queue. Then the `Python` program received the packet from the queue, extracted the `HTTP request line`, performed normalization, and checked it against predefined rules. If the request was classified as `BENIGN`, the packet was allowed. If indicators of `path traversal`, `XSS-like`, or `SQLi-like payload` were detected, the packet was blocked.

Component	Purpose
Linux endpoint	Protected endpoint system
Apache HTTP Server	Local HTTP service on port 80
iptables	Redirects incoming HTTP traffic to <code>NFQUEUE</code>
NFQUEUE	Passes packets from the Linux kernel to a user-space <code>Python</code> program
Python HTTP Guard Agent	Analyzes the <code>request line</code> , normalizes it, and makes an <code>ACCEPT/DROP</code> decision
CSV logs	Structured event recording
Agent API	Local <code>read-only</code> API for the dashboard
SIEM-like Dashboard	Visualization of metrics, events, <code>self-test</code> , and <code>validation</code>
Windows attacker client	External client for sending <code>BENIGN</code> and <code>suspicious</code> HTTP requests

Table 3.1: Components of the laboratory environment

The final project structure includes source code, configuration files, a test dataset, experiment results, screenshots, and documentation. This improves reproducibility because all key components are located in one project folder and can be inspected separately.

Architecturally, the system is divided into four layers: the source of HTTP requests, the protected endpoint, the inspection and monitoring layer, and the results layer. Requests can be sent either from a local `Linux` client or from a separate `Windows attacker client`. Protection is performed on the `Linux endpoint`; the `dashboard` does not block traffic and only displays the results produced by `HTTP Guard`.

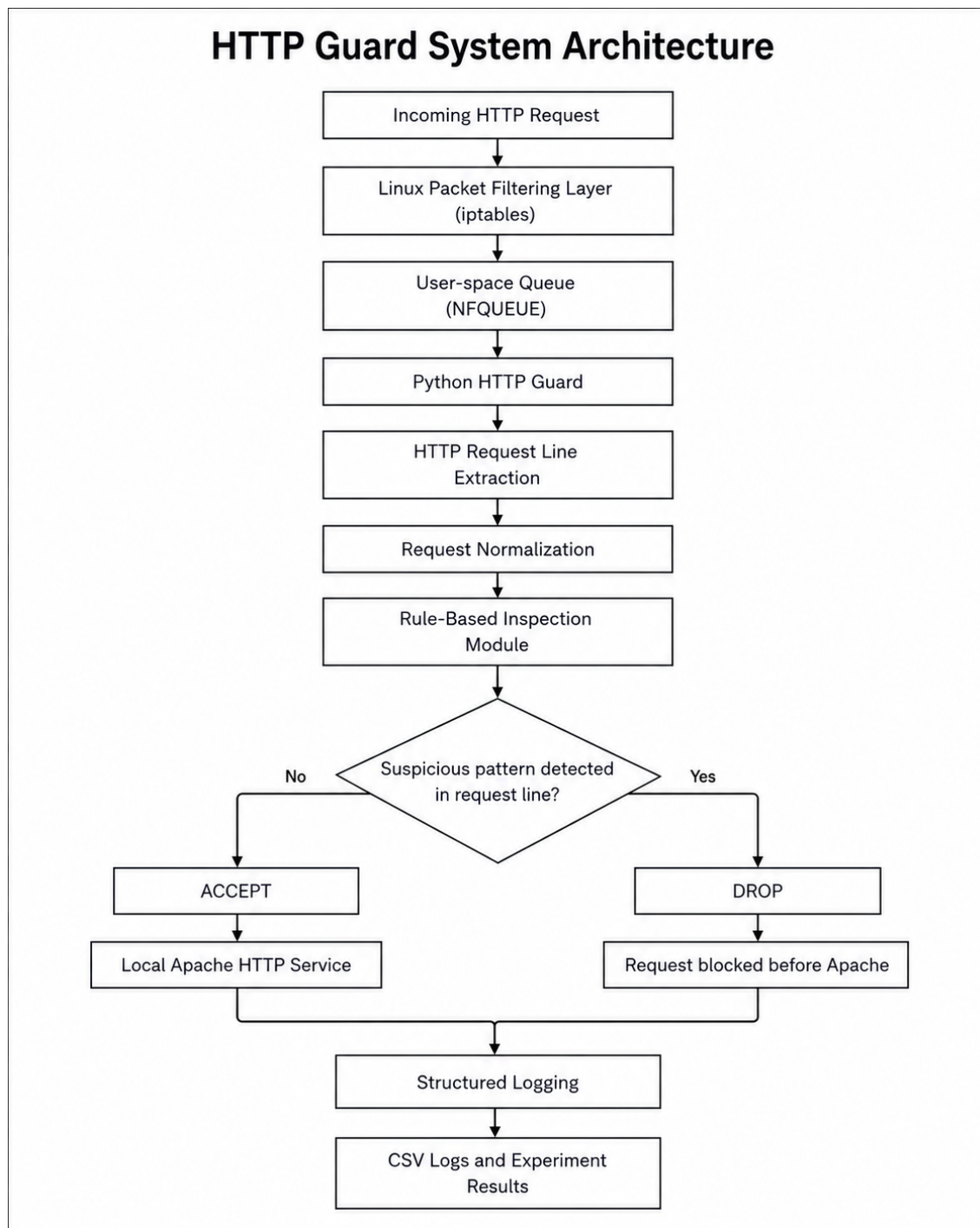


Figure 3.1: Simplified architecture of the proposed HTTP Guard mechanism.

3.2 Implementation of the HTTP Guard Agent

The main component of the practical part is the **HTTP Guard Agent**, implemented in **Python**. Its task is to process HTTP packets received from **NFQUEUE** and decide what should happen to each packet. The agent operates in **user-space**, but it receives packets through an **iptables** rule that redirects incoming HTTP traffic to **NFQUEUE**.

The agent logic includes the following stages:

1. receiving an IP packet from **NFQUEUE**;
2. checking the presence of a TCP layer and destination port 80;
3. extracting the first line of the HTTP request;
4. normalizing the **request line**;
5. checking the **raw** and **normalized request line** against **rule-based** indicators;
6. assigning a request category;
7. making an **ACCEPT** or **DROP** decision;
8. writing the event to a structured **CSV** log;
9. returning the decision to **NFQUEUE**.

The analyzed object is the **HTTP request line**. This choice is justified because many demonstrative variants of **path traversal**, **XSS**, and **SQL injection** appear in the **URI** and **query** parameters. For example, the request line may contain the sequence **../**, an **encoded script tag**, or an **SQL-like** construction such as **' OR 1=1--**.

A key implementation feature is normalization. At an earlier stage of the project, one test request with **encoded path traversal** was not blocked correctly because the malicious sequence was not present in direct form. To address this limitation, repeated **URL** decoding and conversion to a unified format were added. This made it possible to detect not only direct payloads but also some encoded variants.

An example of encoded sequence normalization is:

`%252e%252e -> %2e%2e -> ..`

After normalization, the request line is converted to lowercase, and backslashes are replaced with forward slashes. This reduces the likelihood of bypassing rules through case variation, `percent-encoding`, or an alternative path separator.

Request categorization is performed using rules stored in a configuration file. If the `request line` matches one of the patterns, the request receives the category `TRAVERSAL`, `XSS`, or `SQLI`. If no match is found, the request is classified as `BENIGN`. For malicious categories, the action is set to `DROP`; for `benign` requests, it is set to `ACCEPT`.

The result of processing each request is stored in a `CSV` log. The log records `timestamp`, `source IP`, `source port`, `destination IP`, `destination port`, `raw request line`, `normalized request line`, `category`, `matched pattern`, and `action`. This structure makes the log useful not only for debugging but also for later visualization in the dashboard.

```
1 # Core HTTP Guard decision logic
2
3 raw_request_line = extract_request_line(ip)
4
5 if not raw_request_line:
6     packet.accept()
7     return
8
9 normalized_request_line = normalize_request_line(raw_request_line)
10
11 category, matched_pattern = detect_pattern(
12     raw_request_line,
13     normalized_request_line,
14     RULES
15 )
16
17 if category == "BENIGN":
18     action = "ACCEPT"
19 else:
20     action = "DROP"
21
22 write_log(
23     src_ip=ip.src,
24     src_port=tcp.sport,
25     dst_ip=ip.dst,
26     dst_port=tcp.dport,
27     raw_request_line=raw_request_line,
28     normalized_request_line=normalized_request_line,
29     category=category,
30     matched_pattern=matched_pattern,
31     action=action
32 )
33
34 if action == "DROP":
35     packet.drop()
36 else:
37     packet.accept()
```

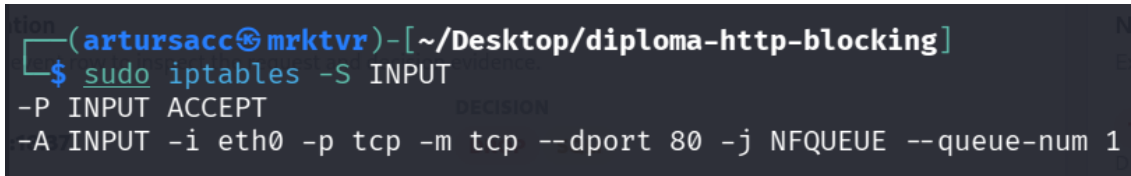
Figure 3.2: Core implementation fragment of the HTTP Guard Agent, including request-line normalization, rule-based detection, structured logging, and the final ACCEPT or DROP decision.

3.3 Configuration of iptables and NFQUEUE

To allow the **HTTP Guard Agent** to make a decision before the request is processed by the web server, incoming HTTP traffic was redirected from the **INPUT** chain to **NFQUEUE** [11, 13, 14]. In the final remote scenario, a rule was used that sent TCP traffic arriving on the network interface **eth0** and port 80 to queue number 1.

The general logic is as follows: a client sends an HTTP request to the protected **Linux endpoint**; the packet enters the **INPUT** chain; the **iptables** rule passes it to **NFQUEUE**. The **Python** program then receives the packet from the queue, analyzes the **request line**, and returns a decision. If the decision is **ACCEPT**, the packet continues processing and reaches **Apache HTTP Service**. If the decision is **DROP**, the packet is discarded before being passed to the web server.

For local testing, a rule on the **loopback** interface was used. For testing with the **Windows attacker client**, a rule on the network interface **eth0** was used. This confirmed that the mechanism works not only for requests to **127.0.0.1**, but also for HTTP requests arriving from another device in the laboratory network.



```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ sudo iptables -S INPUT
-P INPUT ACCEPT
-A INPUT -i eth0 -p tcp -m tcp --dport 80 -j NFQUEUE --queue-num 1
```

Figure 3.3: Input-chain **iptables** rule redirecting incoming TCP traffic on port 80 to **NFQUEUE**, allowing the **HTTP Guard Agent** to inspect requests before they reach **Apache**.

Helper **shell** scripts were prepared in the project folder. The script **setup_iptables.sh** is used to apply the rule that redirects HTTP traffic to **NFQUEUE**, while **clear_iptables.sh** removes rules after the experiments. This reduces the risk of manual errors and makes prototype startup more reproducible.

It is important to note that **NFQUEUE** is used as a mechanism for passing packets from the **Linux kernel** to **user-space**. This enables the implementation of analysis logic in **Python** without writing a custom **kernel module**. For this diploma project, this approach is practical because it allows the work to focus on HTTP request analysis, logging, and experimental evaluation.

3.4 Implementation of the Agent API

To connect the practical component with the `dashboard`, a local `Agent API` was implemented. It operates as a `read-only` access layer for the results and logs of `HTTP Guard`. The `Agent API` does not block traffic and does not replace the `HTTP Guard Agent`. Its purpose is to provide the `dashboard` with structured access to the current project state, logs, metrics, and validation results.

Endpoint	Purpose
<code>/api/status</code>	Checks <code>Agent API</code> availability and basic project state
<code>/api/metrics</code>	Returns aggregated metrics from <code>final results</code> and <code>runtime logs</code>
<code>/api/logs</code>	Returns events from <code>logs/http_guard_events.csv</code>
<code>/api/files</code>	Checks the availability of key project files
<code>/api/validation/checks</code>	Returns the set of <code>quick validation checks</code>
<code>/api/validation/run</code>	Starts the <code>quick live self-test</code>
<code>/api/validation/run-full</code>	Starts full <code>50-test live validation</code>

Table 3.2: Main local `Agent API` endpoints

An important design decision was the separation of component responsibilities. The `HTTP Guard Agent` analyzes and blocks packets. The `Agent API` only reads files, aggregates data, and starts validation checks on user request. The `dashboard` presents the received information in a convenient form but does not make independent blocking decisions.

For safety and predictability, the `Agent API` was designed for a local laboratory environment. Validation requests are limited to local addresses so that the `dashboard` cannot become a tool for arbitrary `HTTP` requests to external networks. This corresponds to the project goal: evaluating `endpoint-level protection` on a controlled `Linux` host.

When the `Agent API` starts, the terminal displays the available `endpoints` and operating mode. This simplifies environment checking before a demonstration: if the `API` is running, the `dashboard` shows the `Online` status, and sections with `live validation` and `runtime logs` can update automatically.

3.5 SIEM-like Dashboard

For result visualization, a local **SIEM-like dashboard** was developed. Its purpose is to present the results of **HTTP Guard** in a form similar to security event monitoring interfaces: final metrics, event log, filtering, search, **investigation** panel, and **evidence** export. The **dashboard** is implemented as a local **HTML** page that can work in static mode or receive live data through the **Agent API**.

The **dashboard** is not a blocking component. It is used as a monitoring and analysis layer. This distinction is essential for the architecture: the **ACCEPT** or **DROP** decision is made by the **HTTP Guard Agent** on the protected **Linux endpoint**, while the **dashboard** only displays already generated data.

Section	Purpose
Final Experiment Metrics	Displays official results of the controlled 50-test dataset
Runtime Guard Log Overview	Aggregates events from the runtime CSV log
Quick Live Protection Self-Test	Performs a quick check with 4 live requests
Full 50-Test Live Validation	Performs full live validation on 50 test cases
SIEM-like Event Explorer	Allows event viewing, filtering, and search
Event Investigation Panel	Provides detailed explanation of a selected event
Normalization Evidence	Shows the raw and normalized request line
Technical Diagnostics	Checks data file availability

Table 3.3: Sections of the **SIEM-like dashboard**

The upper part of the **dashboard** displays basic information about the protected **endpoint**, protected service, inspection chain, data status, **Agent API** availability, and number of **runtime events**. A separate **Final Experiment Metrics** block shows the official results of the controlled experiment: 50 **test cases**, 100% **pass rate**, 20 **ACCEPT**, 30 **DROP**, 0 **false positives**, and 0 **false negatives**.

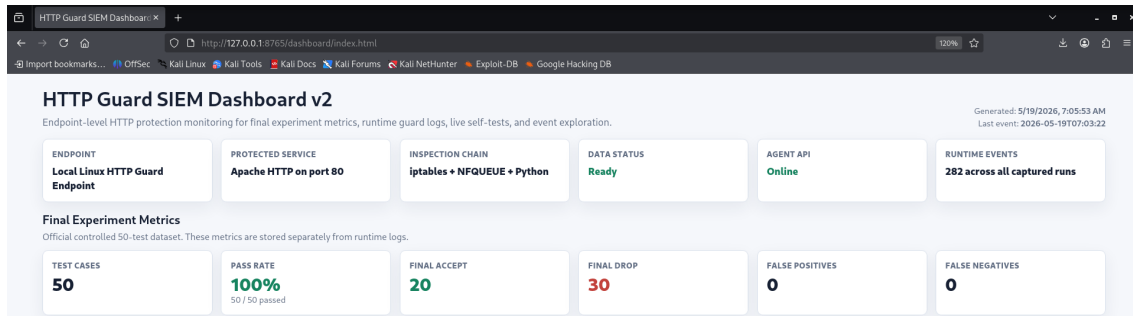


Figure 3.4: Dashboard view of the official controlled experiment metrics: 50 test cases, 100% pass rate, 20 accepted benign requests, 30 dropped malicious requests, and zero false positives or false negatives.

The Runtime Guard Log Overview differs from final experiment metrics. It is built from the current `logs/http_guard_events.csv` file and may therefore include not only the official experiment but also repeated runs, manual checks, quick self-test, full validation, and requests from the Windows attacker client. Therefore, the number of runtime events may be greater than 50. This is expected and reflects the cumulative nature of an operational event log.

The SIEM-like Event Explorer supports event analysis similar to a SIEM workflow: filtering by category, action, pattern, source/destination, and request line. Pagination inspired by Splunk was also added: the user can select the number of rows per page and move between pages of results. Export of filtered events to CSV/JSON is also supported, which is useful for preparing evidence for the report.

When a row is selected in the Event Explorer, an investigation panel opens. It displays timestamp, source, destination, raw request line, normalized request line, matched pattern, category, decision, and a brief explanation of the classification. As a result, the dashboard shows not only that blocking occurred but also why the decision was made.

The Normalization Evidence section was added to demonstrate how an encoded payload is transformed into a normalized form. This is especially important for path traversal payloads, where characters may be represented through percent-encoding. In the report, this section is used as evidence that a limitation of an earlier prototype version was addressed in part through request normalization.

3.6 Data Collection Methodology and Test Dataset

For experimental evaluation, a controlled set of 50 HTTP `test cases` was prepared. Each test case contains a request path, category, and expected action. This makes it possible to compare the actual HTTP Guard decision with the predefined `expected result` and calculate final quality indicators.

Category	Purpose	Expected action
BENIGN	Ordinary HTTP requests without malicious indicators	ACCEPT
TRAVERSAL	Requests with <code>path traversal</code> indicators	DROP
XSS	Requests with <code>XSS-like payload</code>	DROP
SQLI	Requests with <code>SQL injection-like payload</code>	DROP

Table 3.4: Categories of test HTTP requests

The `benign` group included ordinary URI paths such as `/`, `/index.html`, `/home`, `/about`, `/contact`, `/products`, `/news`, `/login`, `/api/status`, and similar paths. For these requests, the expected result is `ACCEPT` because they do not contain malicious indicators.

The `suspicious` groups included requests with characteristic attack patterns. For `path traversal`, variants with `../`, encoded representations, and attempts to access system or configuration files were used. For `XSS-like` tests, strings with `<script>`, `javascript:`, and event handlers such as `onerror=` were used. For `SQLi-like` tests, constructions with a single quote, `OR 1=1`, `UNION SELECT`, and `SQL` comments were used.

File	Purpose
<code>results/final_experiment_results.csv</code>	Raw case results of 50 <code>test cases</code>
<code>results/final_experiment_summary.json</code>	Summary statistics of the experiment

Table 3.5: Result files of the controlled experiment

In addition, `runtime` events were saved in `logs/http_guard_events.csv`. This file is used to analyze actual runs, `manual checks`, and `live validation`; therefore, its number of records may differ from the official 50 `test cases`.

The evaluation methodology is based on comparing two values: `expected action` and `observed action`. If they match, the test is considered passed. If a `benign` request

is blocked, it is counted as a **false positive**. If a **suspicious** request is allowed, it is counted as a **false negative**. This evaluation format is convenient for demonstrating prototype correctness on a limited but controlled request set.

3.7 Controlled Experiment on 50 HTTP Test Cases

After normalization was implemented and rules were updated, a **controlled experiment** was conducted on 50 **HTTP test cases**. The results showed that all test requests were processed according to their expected actions.

Metric	Value
Total test cases	50
Passed	50
Pass rate	100%
Final ACCEPT	20
Final DROP	30
False positives	0
False negatives	0

Table 3.6: Final indicators of the **controlled experiment**

The obtained result shows that, on the prepared test set, the prototype correctly distinguishes between **benign** and **suspicious HTTP request lines**. Benign requests were allowed, while requests with **path traversal**, **XSS-like**, and **SQLi-like payload** indicators were blocked.

A separate **full live validation** was also performed through the **dashboard**. Unlike the static **controlled experiment**, **live validation** sends real HTTP requests to the protected **endpoint** and compares the **observed result** with the expected action. For **suspicious** requests, blocking appears as a **timeout** on the client side because the packet is dropped by **HTTP Guard** before it reaches **Apache**.

CHAPTER 3. PROTOTYPE IMPLEMENTATION AND EXPERIMENTAL EVALUATION

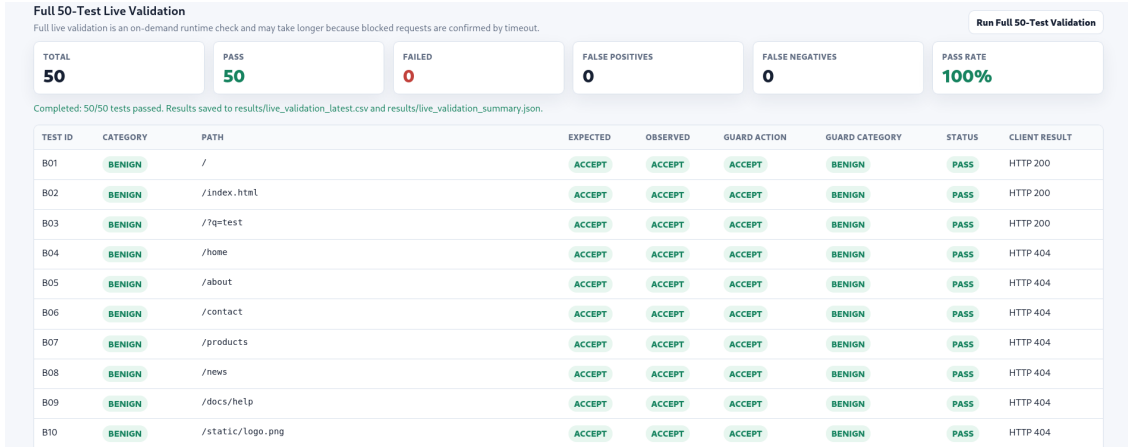


Figure 3.5: Full 50-test live validation result displayed in the dashboard, confirming that all prepared HTTP test cases produced the expected decisions.

Full 50-Test Live Validation also showed 50 successful checks out of 50. The dashboard displayed expected action, observed result, guard action, guard category, and client result. This makes it possible to view not only the final statistics but also per-case validation results.

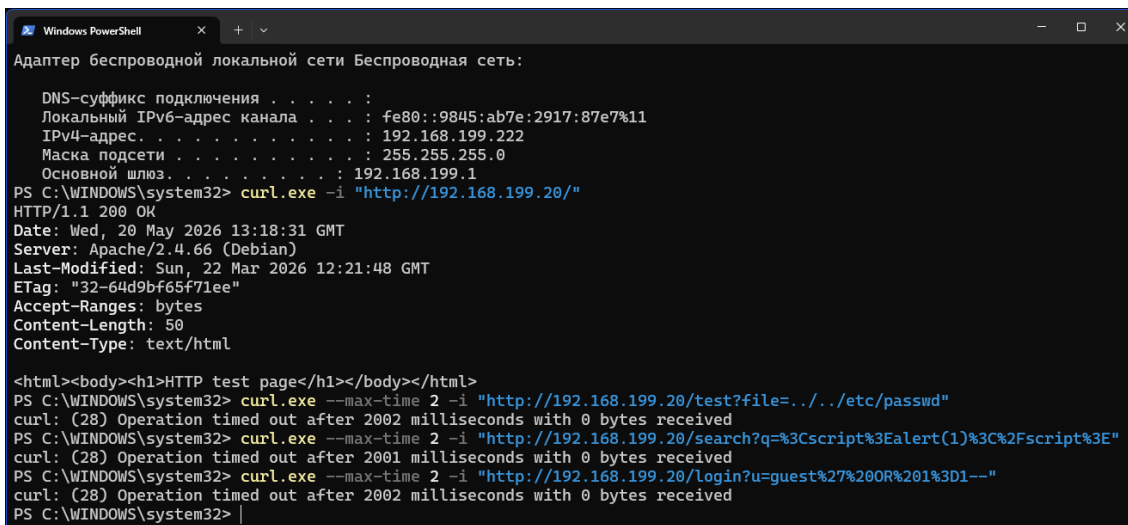
It is important to distinguish controlled final metrics from runtime log overview. Final metrics are based on the official set of 50 test cases. The Runtime log overview is based on the cumulative event log and may include repeated checks, quick checks, manual checks, the Windows attacker scenario, and repeated tests. Therefore, the runtime event count may be greater than 50, and this is not an error.

3.8 Verification of the Windows Attacker Client to Linux Protected Endpoint Scenario

To demonstrate that the mechanism works not only on local loopback traffic, a scenario with a separate Windows attacker client was tested. In this scenario, the Windows machine sent HTTP requests to the protected Linux endpoint over the local network. The Linux endpoint had the IP address 192.168.199.20, while the Windows attacker client had 192.168.199.222.

The verification scenario included the following stages:

1. Apache HTTP Server was started on the Linux endpoint;
2. an iptables rule was added to redirect incoming HTTP traffic on eth0:80 to NFQUEUE;
3. the Python HTTP Guard Agent was started;
4. benign and suspicious HTTP requests were sent from Windows PowerShell;
5. the results were checked in the Guard terminal and in the SIEM-like dashboard.



```
Windows PowerShell
Адаптер беспроводной локальной сети Беспроводная сеть:

DNS-суффикс подключения . . . . . :
Локальный IPv6-адрес канала . . . . : fe80::9845:ab7e:2917:87e7%11
IPv4-адрес . . . . . : 192.168.199.222
Маска подсети . . . . . : 255.255.255.0
Основной шлюз . . . . . : 192.168.199.1
PS C:\WINDOWS\system32> curl.exe -i "http://192.168.199.20/"
HTTP/1.1 200 OK
Date: Wed, 20 May 2026 13:18:31 GMT
Server: Apache/2.4.66 (Debian)
Last-Modified: Sun, 22 Mar 2026 12:21:48 GMT
ETag: "32-64d9bf65f71ee"
Accept-Ranges: bytes
Content-Length: 50
Content-Type: text/html

<html><body><h1>HTTP test page</h1></body></html>
PS C:\WINDOWS\system32> curl.exe --max-time 2 -i "http://192.168.199.20/test?file=../../etc/passwd"
curl: (28) Operation timed out after 2002 milliseconds with 0 bytes received
PS C:\WINDOWS\system32> curl.exe --max-time 2 -i "http://192.168.199.20/search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E"
curl: (28) Operation timed out after 2001 milliseconds with 0 bytes received
PS C:\WINDOWS\system32> curl.exe --max-time 2 -i "http://192.168.199.20/login?u=guest%27%20OR%201%3D1--"
curl: (28) Operation timed out after 2002 milliseconds with 0 bytes received
PS C:\WINDOWS\system32> |
```

Figure 3.6: Windows PowerShell commands used to send benign and suspicious HTTP requests to the protected Linux endpoint during the remote demonstration scenario.

The results confirmed the expected behavior. A normal request to the root path / received HTTP 200 OK, meaning that it was allowed. Requests with path traversal, XSS-like, and SQLi-like payload indicators ended with a timeout on the Windows client because HTTP Guard made a DROP decision.

On the Linux endpoint, the HTTP Guard terminal recorded incoming packets from source IP 192.168.199.222 to destination IP 192.168.199.20:80. For each request, the agent displayed the raw request line, normalized request line, category, matched pattern, and action.

```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ sudo python3 proto/nfqueue_http_guard.py
[sudo] password for artursacc:
[*] HTTP Guard started
[*] Project directory: /home/artursacc/Desktop/diploma-http-blocking
[*] Rules file: /home/artursacc/Desktop/diploma-http-blocking/config/rules.json
[*] Log file: /home/artursacc/Desktop/diploma-http-blocking/logs/http_guard_events.csv
[*] Loaded rule categories: ['TRAVERSAL', 'XSS', 'SQLI']
[*] Waiting for HTTP packets in NFQUEUE 1...

[+] Packet: 192.168.199.222:4969 → 192.168.199.20:80
[+] Raw request line: GET / HTTP/1.1
[+] Normalized request line: get / http/1.1
[+] Category: BENIGN
[+] Action: ACCEPT

[+] Packet: 192.168.199.222:4980 → 192.168.199.20:80
[+] Raw request line: GET /test?file=../../../../etc/passwd HTTP/1.1
[+] Normalized request line: get /test?file=../../../../etc/passwd http/1.1
[+] Category: TRAVERSAL
[!] Matched pattern: ../../
[+] Action: DROP

[+] Packet: 192.168.199.222:4998 → 192.168.199.20:80
[+] Raw request line: GET /search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E HTTP/1.1
[+] Normalized request line: get /search?q=<script>alert(1)</script> http/1.1
[+] Category: XSS
[!] Matched pattern: <script
[+] Action: DROP

[+] Packet: 192.168.199.222:5006 → 192.168.199.20:80
[+] Raw request line: GET /login?u=guest%27%20OR%201%3D1-- HTTP/1.1
[+] Normalized request line: get /login?u=guest' or 1=1-- http/1.1
[+] Category: SQLI
[!] Matched pattern: ' or 1=1
[+] Action: DROP
```

Figure 3.7: HTTP Guard terminal output on the protected Linux endpoint, showing events generated by requests from the Windows attacker client and the corresponding ACCEPT or DROP actions.

The same events were displayed in the SIEM-like Event Explorer of the dashboard. This confirmed that the event log is populated correctly not only during local tests but also when requests arrive from another device in the laboratory network.

The investigation panel was additionally checked for an SQLi-like event. It showed the original request line, normalized request line, matched pattern ' or 1=1, category SQLI, and decision DROP. This is useful for explaining the result during defense because it demonstrates not only blocking but also the reason for blocking.



Figure 3.8: Dashboard investigation panel for a SQL injection-like event, including the raw request line, normalized request line, matched pattern, category, action, and explanation.

Thus, the Windows Attacker Client to Linux Protected Endpoint scenario confirmed that the developed mechanism is applicable to incoming network HTTP traffic, not only to local test requests.

3.9 Analysis of Results

The experimental evaluation showed that the prototype fulfilled the task within the selected laboratory model. On the controlled dataset of 50 test cases, the system achieved a 100% pass rate, with no false positives or false negatives. This means that, within the prepared dataset, benign requests were not blocked, and suspicious requests did not reach the HTTP service.

The main practical result is the confirmation of endpoint-level interception. Traffic is intercepted on the Linux endpoint before it reaches Apache. This is visible because malicious requests end with a client-side timeout, while the HTTP Guard log records the DROP action. At the same time, benign requests pass through and receive a response from Apache.

An important improvement compared with an earlier version of the prototype was normalization of the HTTP request line. Previously, encoded path traversal could be handled incorrectly because the malicious sequence was not always present in direct

form. After repeated URL decoding and conversion to a unified format were added, encoded variants became more detectable by the **rule-based** logic.

The **SIEM-like dashboard** strengthened the practical part of the project by presenting the mechanism in a form suitable for analysis. Instead of relying only on **terminal output**, the user obtains **summary metrics**, **runtime counters**, category filters, **event investigation**, and **export evidence**. This makes the prototype closer to real security monitoring processes, where it is important not only to block an event but also to explain why it was classified as **suspicious**.

The results should be interpreted correctly. The 100% value refers to the controlled test set of 50 prepared HTTP requests. It does not mean absolute protection against all possible web attacks. The **rule-based** approach works well for known patterns but may require rule expansion and more advanced normalization for new bypass variants.

It is also important to distinguish between the number of **final test cases** and the number of **runtime events**. The **runtime log** accumulates events from different runs: **controlled validation**, **quick checks**, **manual checks**, the **Windows attacker scenario**, and repeated tests. Therefore, **runtime counters** are used for monitoring activity, while **final experiment metrics** are used for formal evaluation of the prepared dataset.

3.10 Limitations and Directions for Further Development

Despite the successful experimental evaluation, the developed prototype has several limitations. These limitations are related both to the selected architecture and to the scope of the diploma project.

The first limitation is support for **plaintext HTTP** only. The prototype analyzes the **HTTP request line**, which is available in unencrypted form. With **HTTPS**, request content is protected by **TLS**; therefore, analysis of the **URI** and **payload** would require additional mechanisms such as **TLS termination**, a **reverse proxy**, or integration with the application before encryption or after decryption.

The second limitation is **rule-based detection**. This approach is understandable,

explainable, and suitable for demonstration, but it depends on the completeness of the rule set. If an attacker uses a new bypass form not covered by the rules or normalization, the request may be classified incorrectly. Therefore, future work should expand the rule set and test a broader range of payload variants.

The third limitation is laboratory scale. Evaluation was performed in a controlled environment: a `Linux endpoint`, `Apache HTTP Server`, a local network, and a predefined set of `test cases`. Industrial use would require performance testing, stability checks under a large number of connections, support for multiple `endpoint` systems, and centralized event storage.

The fourth limitation is simplified HTTP processing. The prototype focuses on the `request line` and does not perform full analysis of the `HTTP body`, complex `multi-packet` scenarios, or all aspects of `TCP reassembly`. This is sufficient for a diploma demonstration, but a `production-level` solution would require deeper protocol processing.

Direction	Description
Expanded normalization	Support for more encoded and obfuscated payload variants
Expanded rule set	New rules for additional attack classes
Larger labeled dataset	Evaluation on a larger labeled set of HTTP requests
Multi-endpoint deployment	Deployment of the agent on several <code>Linux endpoint</code> systems
HTTPS/TLS scenario	Investigation of protected traffic analysis options
Performance testing	Evaluation of latency and stability under high load
SIEM integration	Forwarding events to an external SIEM system
ML-based detection	Possible extension after collecting a sufficient dataset

Table 3.7: Directions for further prototype development

`ML-based detection` is considered in this work only as a direction for future development. Correct use of ML requires a large labeled `dataset`, model training, validation, and a separate quality evaluation. In the current diploma project, the main emphasis was placed on an explainable `rule-based` mechanism that can be demonstrated and verified transparently.

The practical result of the work is a functioning prototype of an `endpoint-level` mechanism for blocking malicious HTTP requests. It includes traffic interception through

iptables and NFQUEUE, the Python HTTP Guard Agent, request line normalization, rule-based detection, structured logging, the Agent API, and the SIEM-like dashboard. Experimental evaluation confirmed correct operation of the mechanism on the controlled dataset of 50 test cases and in a scenario with an external Windows attacker client.

Conclusion

During the thesis work, a security mechanism for blocking malicious HTTP requests on an endpoint system was developed and experimentally evaluated. The main idea of the work was to analyze incoming HTTP traffic directly on a `Linux endpoint` before the request is passed to the local `Apache` web service.

The work examined the main types of malicious HTTP requests, including `path traversal`, `XSS-like payload`, and `SQL injection-like payload`. Based on this analysis, a `rule-based` detection model was selected because it makes the decision explainable through the request category and the `matched pattern`. To improve detection of encoded payload variants, normalization of the `HTTP request line` was implemented.

The practical implementation combines `iptables`, `NFQUEUE`, and the `Python HTTP Guard Agent`. An `iptables` rule redirects incoming HTTP traffic to `NFQUEUE`. The `Python` program then extracts the `request line`, performs normalization, checks the request against the rules, and makes an `ACCEPT` or `DROP` decision. For every processed event, a `structured CSV log` is created with the `source/destination`, `raw request line`, `normalized request line`, `category`, `matched pattern`, and `action`.

For result monitoring, a local `Agent API` and a `SIEM-like dashboard` were developed. The `dashboard` displays the final metrics of the `controlled experiment`, `runtime log overview`, `quick self-test`, `full 50-test live validation`, `SIEM-like event explorer`, `investigation panel`, `normalization evidence`, and `export evidence`. This allowed the mechanism to be presented not only as a `terminal-based prototype`, but also as a system with elements of security event monitoring.

The experimental evaluation was conducted on a `controlled dataset` of 50 HTTP `test cases`. The results showed 50 successful checks out of 50, a `pass rate` of 100%, 20 accepted `benign` requests, 30 blocked `suspicious` requests, 0 `false positives`, and

0 false negatives. In addition, full live validation confirmed correct operation when real HTTP requests were sent to the protected endpoint.

The Windows Attacker Client to Linux Protected Endpoint scenario was also verified. In this scenario, a Windows machine sent benign and suspicious HTTP requests to a Linux endpoint over the local network. The benign request was accepted and received an Apache response, while path traversal, XSS-like, and SQLi-like requests were blocked by HTTP Guard. The events were recorded both in the Guard terminal and in the SIEM-like dashboard, including the source IP of the external client.

Thus, the aim of the thesis was achieved. The developed prototype demonstrates the complete cycle of endpoint-level protection: HTTP request interception, normalization, rule-based detection, decision-making, logging, and visualization of results. The obtained results confirm the effectiveness of the proposed mechanism within the controlled laboratory environment and the prepared test dataset.

At the same time, the prototype has several limitations. It is focused on plaintext HTTP, analyzes primarily the request line, uses rule-based detection, and was evaluated in a laboratory environment. Further development may include expanding the rule set, improving normalization, evaluating a larger dataset, supporting multi-endpoint deployment, studying HTTPS/TLS scenarios, conducting performance testing, integrating with external SIEM systems, and investigating ML-based detection after a sufficiently large labeled dataset has been collected.

The practical significance of the work is that the created prototype can be used as a laboratory example of an endpoint-level HTTP request blocking mechanism and as a basis for further research into local traffic filtering, event logging, and security event monitoring.

Bibliography

- [1] Roy T. Fielding, Mark Nottingham, and Julian Reschke.
HTTP Semantics.
Technical Report RFC 9110, RFC Editor, 2022.
Accessed: 2026-05-20.
- [2] Roy T. Fielding, Mark Nottingham, and Julian Reschke.
HTTP/1.1.
Technical Report RFC 9112, RFC Editor, 2022.
Accessed: 2026-05-20.
- [3] OWASP Foundation.
Path Traversal.
https://owasp.org/www-community/attacks/Path_Traversal, 2026.
Accessed: 2026-05-20.
- [4] OWASP Foundation.
Testing Directory Traversal File Include.
https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/05-Authorization_Testing/01-Testing_Directory_Traversal_File_Include, 2026.
Accessed: 2026-05-20.
- [5] PortSwigger Web Security Academy.
What is path traversal, and how to prevent it?
<https://portswigger.net/web-security/file-path-traversal>, 2026.
Accessed: 2026-05-20.
- [6] OWASP Foundation.
Cross Site Scripting (XSS).

- <https://owasp.org/www-community/attacks/xss/>, 2026.
Accessed: 2026-05-20.
- [7] OWASP Foundation.
Cross Site Scripting Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html, 2026.
Accessed: 2026-05-20.
- [8] MDN Web Docs.
Cross-site scripting (XSS).
<https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS>, 2026.
Accessed: 2026-05-20.
- [9] OWASP Foundation.
SQL Injection Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html, 2026.
Accessed: 2026-05-20.
- [10] OWASP Foundation.
Injection Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html, 2026.
Accessed: 2026-05-20.
- [11] Netfilter Project.
libnetfilter_queue Documentation.
https://netfilter.org/projects/libnetfilter_queue/doxygen/html/, 2026.
Accessed: 2026-05-20.
- [12] Netfilter Project.
The netfilter.org libnetfilter_queue Project.
https://www.netfilter.org/projects/libnetfilter_queue/index.html, 2026.
Accessed: 2026-05-20.
- [13] Linux man-pages project.

iptables(8) Linux Manual Page, 2026.

Accessed: 2026-05-20.

[14] Netfilter Project.

iptables-extensions(8) Manual Page, 2026.

Accessed: 2026-05-20.

Appendix A

Architectural Diagrams

This appendix presents diagrams that explain the architecture of the developed mechanism and the general algorithm for processing an HTTP request.

General System Architecture

File: 24-architecture-diagram.png

The simplified architecture diagram shows the request path from an incoming HTTP request to endpoint-level filtering and logging. The packet is first processed by the Linux packet filtering layer, redirected to NFQUEUE, and inspected by the Python HTTP Guard agent. The agent extracts and normalizes the HTTP request line, applies rule-based inspection, and makes an ACCEPT or DROP decision. Accepted requests are forwarded to the local Apache HTTP Service, while suspicious requests are blocked before Apache. In both cases, the decision is written to structured logs and experiment result files.

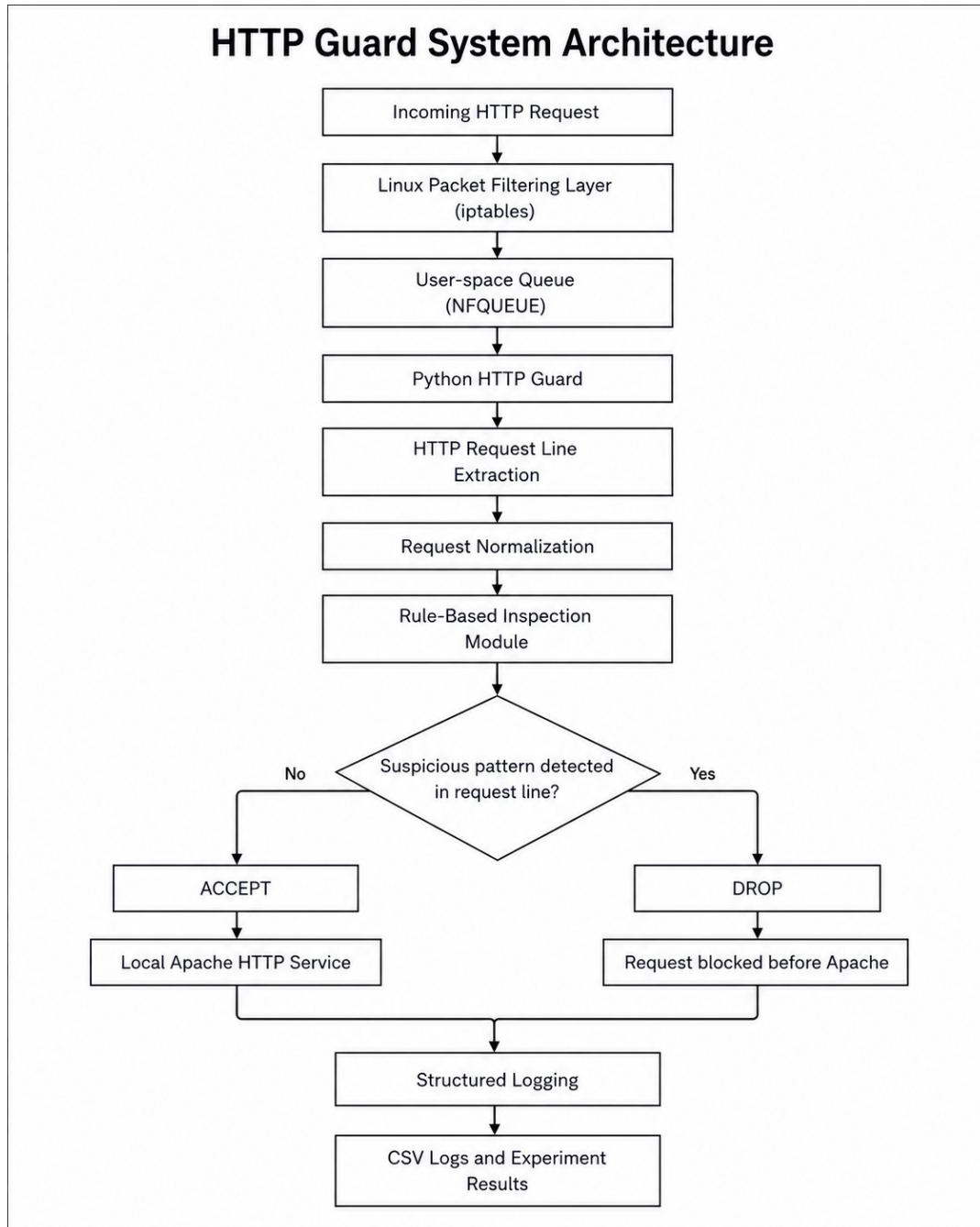


Figure 1: Simplified architecture of the HTTP Guard mechanism.

Request Processing Algorithm Flowchart

The flowchart shows the packet processing sequence: receiving a packet from NFQUEUE, checking TCP and destination port 80, extracting the HTTP request line, checking the supported HTTP method, normalizing the string, checking the raw and normalized

request line against rules, assigning a category, making an ACCEPT or DROP decision, writing a structured log, and returning the decision to NFQUEUE.

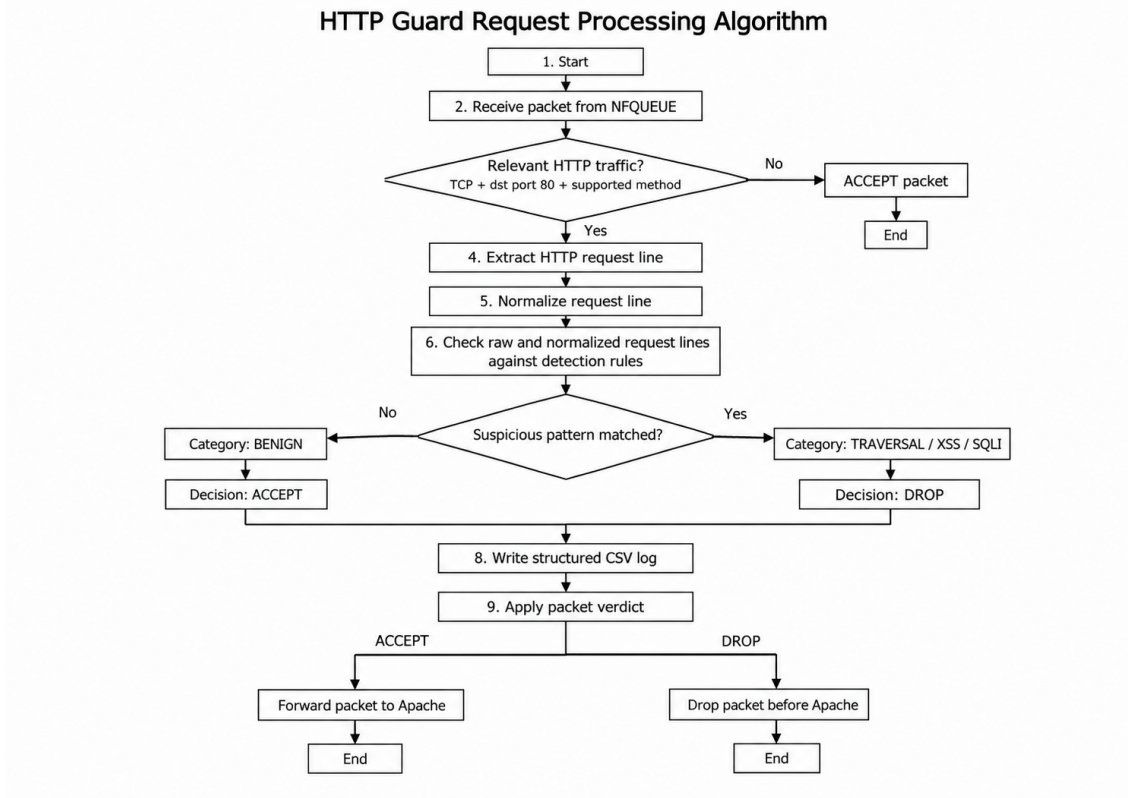


Figure 2: Request processing algorithm of the HTTP Guard prototype.

Appendix B

Screenshots of the Practical Implementation

This appendix contains final screenshots of the practical part of the project, confirming the operation of the developed prototype.

Final Project Structure

File: 31-project-structure-final.png

The screenshot demonstrates the final project structure. The project contains the folders `agent_api`, `dashboard`, `config`, `docs`, `logs`, `proto`, `results`, `scripts`, `tests`, and `screenshots`. This structure shows that the practical part is organized as a reproducible project rather than a single script.

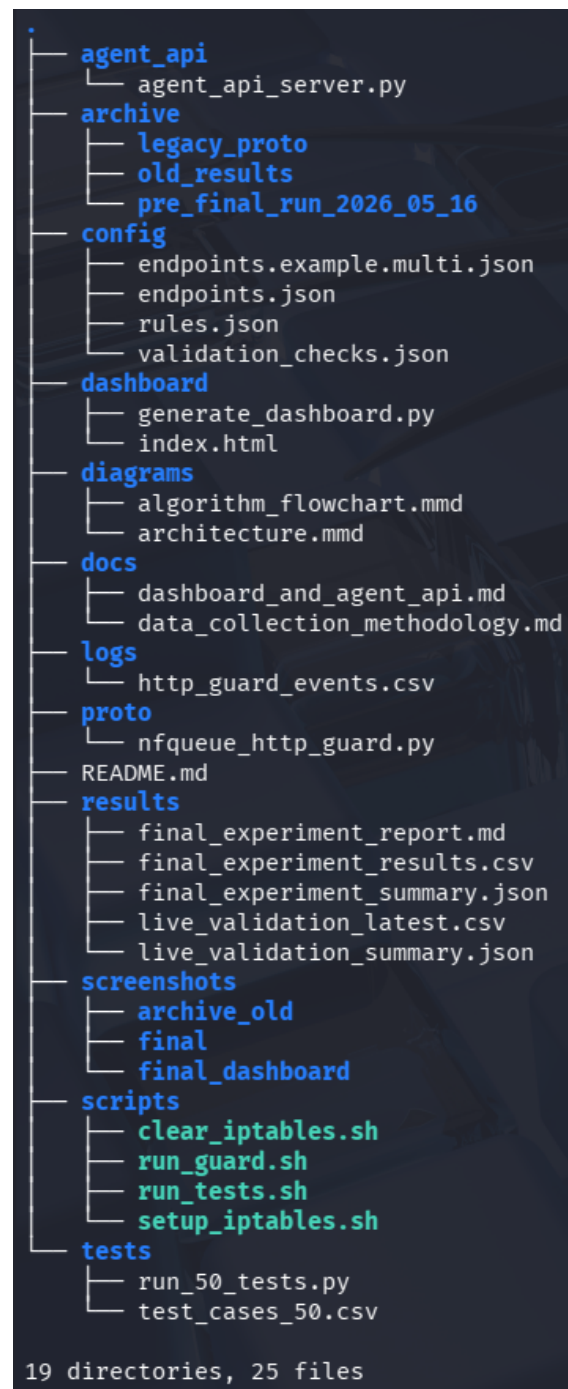


Figure 3: Final project structure of the practical implementation.

Controlled Experiment Metrics

File: 33-dashboard-final-experiment-metrics.png

The screenshot shows the Final Experiment Metrics block in the dashboard. It displays 50 test cases, a pass rate of 100%, 20 ACCEPT, 30 DROP, 0 false positives, and 0 false negatives. This figure confirms the final evaluation of the controlled dataset.

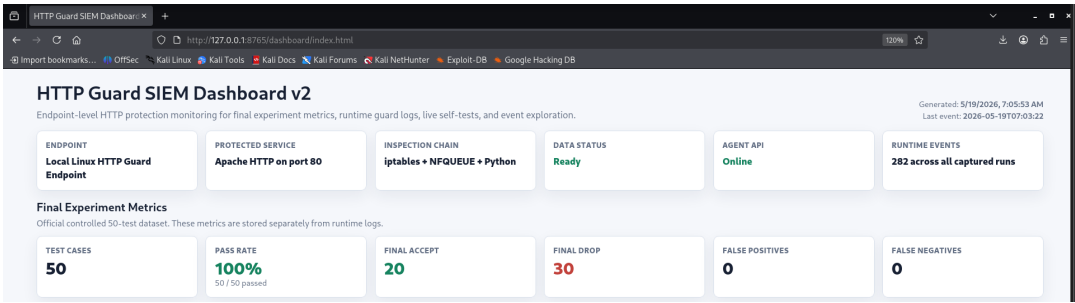


Figure 4: Dashboard metrics for the final controlled experiment.

Full 50-Test Live Validation

File: 34-full-50-live-validation-100.png

The screenshot shows the result of full live validation on 50 HTTP requests. The dashboard displays total 50, pass 50, failed 0, and a pass rate of 100%. The expected and observed results for individual test cases are also visible.

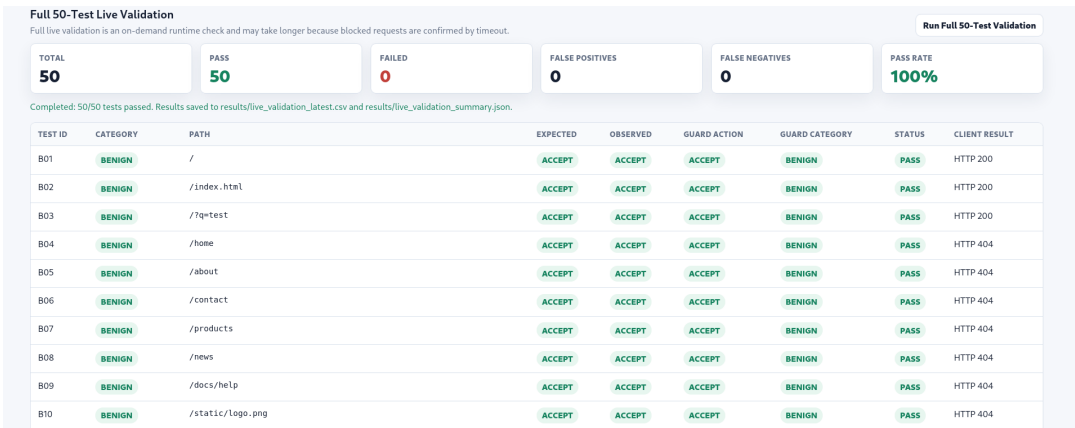
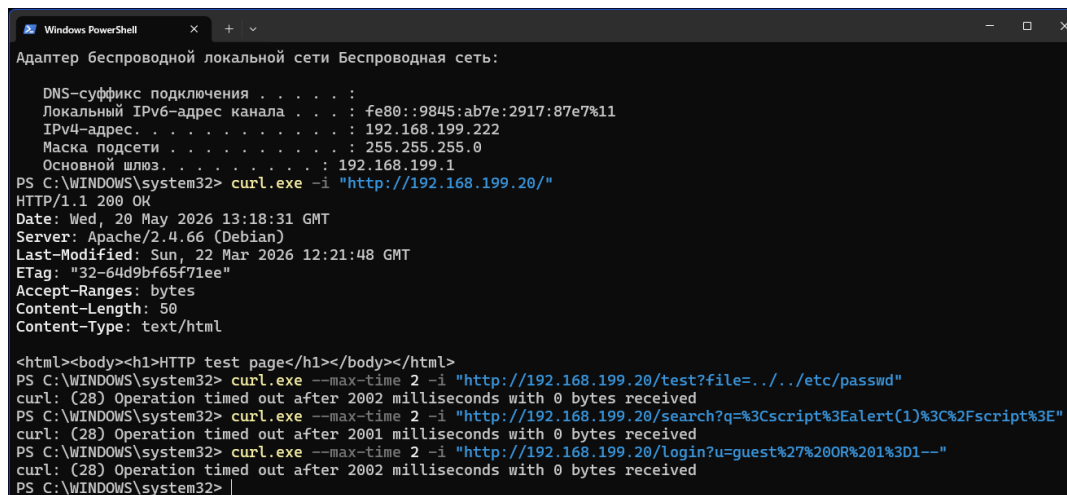


Figure 5: Full live validation result for the 50 HTTP test cases.

Windows Attacker Client

File: 35-windows-attacker-curl-commands.png

The screenshot shows benign and suspicious HTTP requests sent from Windows PowerShell to the protected Linux endpoint. The benign request receives HTTP 200 OK, while suspicious requests end with a timeout, which corresponds to blocking by HTTP Guard.



```

Windows PowerShell
Адаптер беспроводной локальной сети Беспроводная сеть:

DNS-суффикс подключения . . . . . :
Локальный IPv6-адрес канала . . . : fe80::9845:ab7e:2917:87e7%11
IPv4-адрес . . . . . : 192.168.199.222
Маска подсети . . . . . : 255.255.255.0
Основной шлюз . . . . . : 192.168.199.1
PS C:\WINDOWS\system32> curl.exe -i "http://192.168.199.20/"
HTTP/1.1 200 OK
Date: Wed, 20 May 2026 13:18:31 GMT
Server: Apache/2.4.66 (Debian)
Last-Modified: Sun, 22 Mar 2026 12:21:48 GMT
ETag: "32-64d9bf65f71ee"
Accept-Ranges: bytes
Content-Length: 50
Content-Type: text/html

<html><body><h1>HTTP test page</h1></body></html>
PS C:\WINDOWS\system32> curl.exe --max-time 2 -i "http://192.168.199.20/test?file=../../etc/passwd"
curl: (28) Operation timed out after 2002 milliseconds with 0 bytes received
PS C:\WINDOWS\system32> curl.exe --max-time 2 -i "http://192.168.199.20/search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E"
curl: (28) Operation timed out after 2001 milliseconds with 0 bytes received
PS C:\WINDOWS\system32> curl.exe --max-time 2 -i "http://192.168.199.20/login?u=guest%27%20OR%201%3D1--"
curl: (28) Operation timed out after 2002 milliseconds with 0 bytes received
PS C:\WINDOWS\system32>

```

Figure 6: Windows client commands used to test the protected endpoint.

HTTP Guard Events from the Windows Client

File: 36-linux-guard-windows-attacker-events.png

The screenshot shows the terminal output of the HTTP Guard Agent on the Linux endpoint. The source IP corresponds to the Windows attacker client, while the destination IP corresponds to the Linux protected endpoint. For each request, category, matched pattern, and action are displayed.

```

(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ sudo python3 proto/nfqueue_http_guard.py
[sudo] password for artursacc:
[*] HTTP Guard started
[*] Project directory: /home/artursacc/Desktop/diploma-http-blocking
[*] Rules file: /home/artursacc/Desktop/diploma-http-blocking/config/rules.json
[*] Log file: /home/artursacc/Desktop/diploma-http-blocking/logs/http_guard_events.csv
[*] Loaded rule categories: ['TRAVERSAL', 'XSS', 'SQLI']
[*] Waiting for HTTP packets in NFQUEUE 1...

[+] Packet: 192.168.199.222:4969 → 192.168.199.20:80
[+] Raw request line: GET / HTTP/1.1
[+] Normalized request line: get / http/1.1
[+] Category: BENIGN
[+] Action: ACCEPT

[+] Packet: 192.168.199.222:4980 → 192.168.199.20:80
[+] Raw request line: GET /test?file=../../etc/passwd HTTP/1.1
[+] Normalized request line: get /test?file=../../etc/passwd http/1.1
[+] Category: TRAVERSAL
[!] Matched pattern: ../
[+] Action: DROP

[+] Packet: 192.168.199.222:4998 → 192.168.199.20:80
[+] Raw request line: GET /search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E HTTP/1.1
[+] Normalized request line: get /search?q=<script>alert(1)</script> http/1.1
[+] Category: XSS
[!] Matched pattern: <script
[+] Action: DROP

[+] Packet: 192.168.199.222:5006 → 192.168.199.20:80
[+] Raw request line: GET /login?u=guest%27%20OR%201%3D1-- HTTP/1.1
[+] Normalized request line: get /login?u=guest' or 1=1-- http/1.1
[+] Category: SQLI
[!] Matched pattern: ' or 1=1
[+] Action: DROP

```

Figure 7: HTTP Guard event output for requests from the Windows client.

SIEM-like Event Explorer

File: 37-siem-event-explorer-pagination-remote-ip.png

The screenshot shows the dashboard Event Explorer with pagination, filters, export buttons, and events from the external Windows client. This confirms that events are written to the log and are available for analysis through the dashboard.

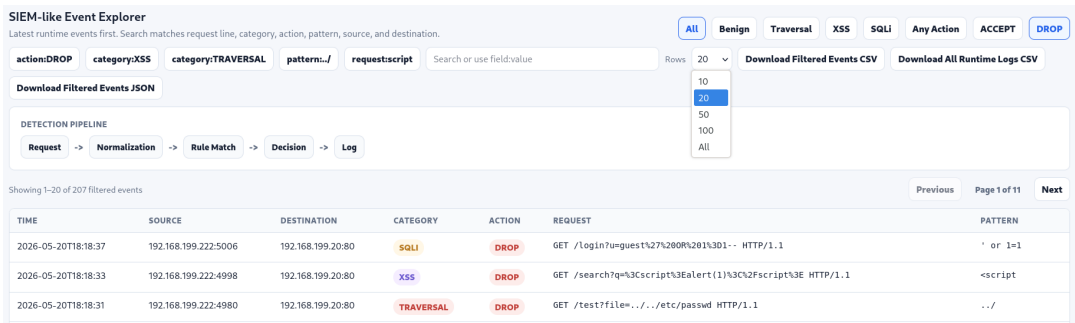


Figure 8: SIEM-like Event Explorer showing remote-client events.

Event Investigation Panel

File: 38-event-investigation-panel-sqli-remote-ip.png

The screenshot shows the investigation panel for an SQLi-like event. The panel displays the raw request line, normalized request line, matched pattern, category, action, and explanation. This demonstrates the explainability of the decision.

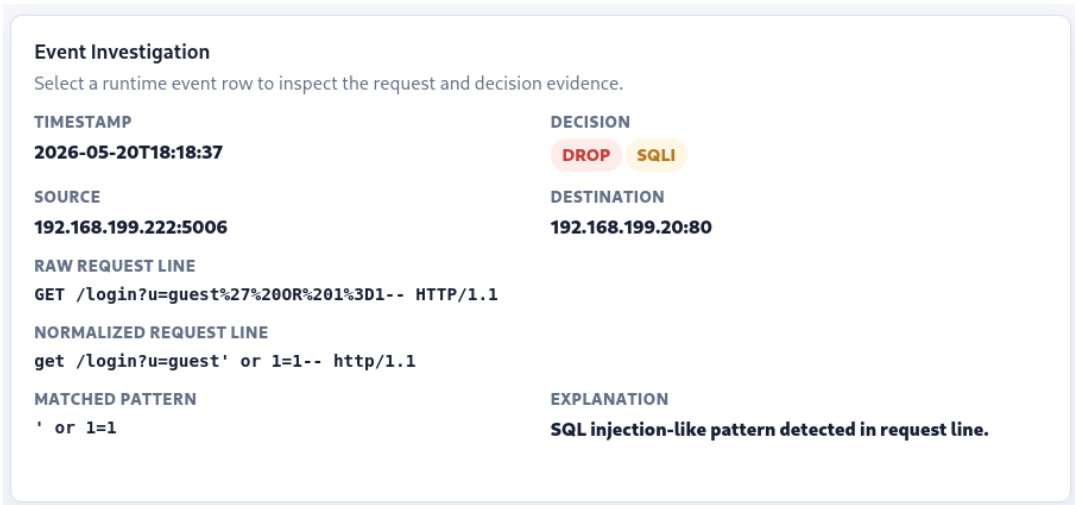


Figure 9: Investigation panel for an SQLi-like request event.

iptables Rule

File: 39-iptables-remote-input-rule.png

The screenshot shows the iptables rule that redirects incoming TCP traffic on eth0:80 to NFQUEUE queue 1. This confirms the mechanism for intercepting traffic

before it reaches Apache.

```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ sudo iptables -S INPUT
-P INPUT ACCEPT
-A INPUT -i eth0 -p tcp -m tcp --dport 80 -j NFQUEUE --queue-num 1
```

Figure 10: iptables rule redirecting incoming HTTP traffic to NFQUEUE.

Agent API and Dashboard Server

Files: 40-agent-api-terminal-running.png, 41-dashboard-server-terminal-running.png

The screenshots demonstrate the launch of the `Agent API` and the local `dashboard server`. The `Agent API` provides the `dashboard` with access to metrics, logs, files, and validation endpoints.

```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ python3 agent_api/agent_api_server.py

[*] HTTP Guard Local Agent API
[*] Listening on http://127.0.0.1:9090
[*] Read-only mode: no iptables, NFQUEUE, sudo, or external traffic
[*] Project directory: /home/artursacc/Desktop/diploma-http-blocking
[*] Endpoints: /api/status /api/metrics /api/logs /api/files
[*] Validation: /api/validation/checks /api/validation/run /api/validation/run-full
```

Figure 11: Agent API terminal output during dashboard support.

```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ python3 -m http.server 8765 --bind 127.0.0.1

Serving HTTP on 127.0.0.1 port 8765 (http://127.0.0.1:8765/) ...
```

Figure 12: Dashboard server terminal output during local monitoring.

Quick Self-Test and Runtime Overview

Files: 42-quick-live-self-test-4-pass.png, 43-runtime-guard-log-overview.png

The screenshots show the `quick live self-test` with four checks and the aggregated overview of `runtime events`. They confirm that the system can perform a quick readiness check and display accumulated events.

Quick Live Protection Self-Test
Quick 4-check validation for live demonstration. Full evaluation is based on the controlled 50-test dataset. Run Quick Self-Test

ENDPOINT Local Linux HTTP Guard Endpoint	PROTECTED URL http://127.0.0.1	QUICK SELF-TEST STATUS Completed 4/4 quick checks passed
---	-----------------------------------	---

TEST ID	CATEGORY	PATH	EXPECTED	OBSERVED	GUARD ACTION	GUARD CATEGORY	STATUS	CLIENT RESULT
live-benign-root Benign root request	BENIGN	/	ACCEPT	ACCEPT	ACCEPT	BENIGN	PASS	HTTP 200
live-traversal Path traversal request	TRAVERSAL	/test?file=../../../../etc/passwd	DROP	DROP	DROP	TRAVERSAL pattern: ..	PASS	TIMEOUT
live-ssx XSS-like request	XSS	/search?q=%3Cscript%3Ealert(1)%3C%2Fscript%3E	DROP	DROP	DROP	XSS pattern: <script	PASS	TIMEOUT
live-sqli SQLi-like request	SQLI	/login?u=admin%27%20OR%201%3D1--	DROP	DROP	DROP	SQLI pattern: ' or 1=1	PASS	TIMEOUT

Figure 13: Quick live self-test result showing successful completion of four readiness checks.

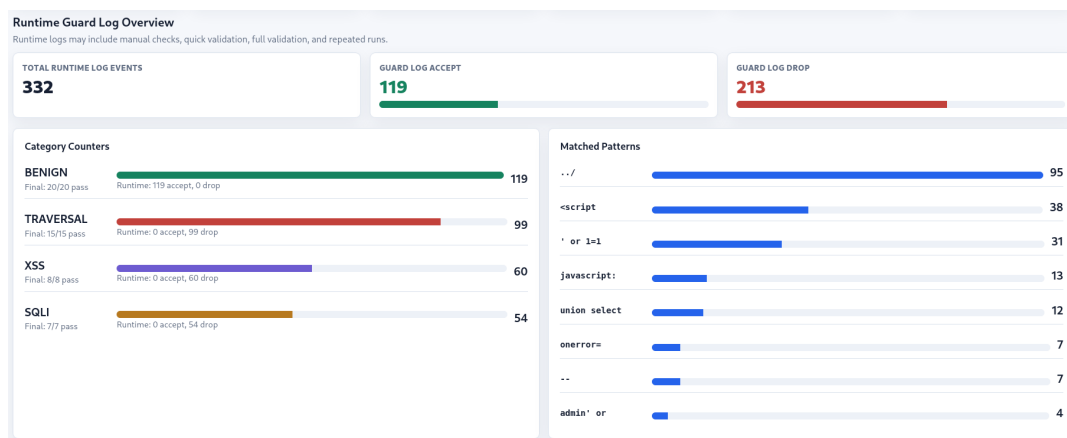


Figure 14: Runtime HTTP Guard log overview showing accumulated event statistics.

Normalization Evidence

File: 44-normalization-evidence.png

The screenshot shows the difference between the raw request line and the normalized request line. It is used as evidence that an encoded payload is transformed into a form suitable for rule-based detection.

Normalization Evidence		
Examples where normalized request lines reveal encoded or transformed payloads.		
TRAVERSAL DROP	Raw: GET /view?path=%252e%252e%252fetc%252fpasswd HTTP/1.1 Normalized: get /view?path=../etc/passwd http/1.1	../
TRAVERSAL DROP	Raw: GET /download?file=%2e%2e%2fetc%2fpasswd HTTP/1.1 Normalized: get /download?file=../etc/passwd http/1.1	../
TRAVERSAL DROP	Raw: GET /view?path=%252e%252e%252fetc%252fpasswd HTTP/1.1 Normalized: get /view?path=../etc/passwd http/1.1	../
TRAVERSAL DROP	Raw: GET /download?file=%2e%2e%2fetc%2fpasswd HTTP/1.1 Normalized: get /download?file=../etc/passwd http/1.1	../

Figure 15: Normalization evidence for an encoded malicious request.

Export Evidence and Diagnostics

Files: 45-export-evidence-buttons.png, 46-technical-diagnostics.png

The screenshots demonstrate export of filtered events to CSV/JSON and verification of key data file availability. This confirms reproducibility and convenience of analysis.



Figure 16: Dashboard controls for exporting filtered evidence.

Technical Diagnostics		
File availability for dashboard and Agent API data sources.		
Guard event log	logs/http_guard_events.csv	332 rows Available
Final experiment summary	results/final_experiment_summary.json	654 bytes Available
Final experiment results	results/final_experiment_results.csv	5559 bytes Available
Controlled 50-test cases	tests/test_cases_50.csv	3244 bytes Available
Endpoint configuration	config/endpoints.json	152 bytes Available
Validation checks	config/validation_checks.json	645 bytes Available

Figure 17: Technical diagnostics panel for required project files.

Result files and structured CSV log

Files: 47-live-validation-result-files.png, 48-guard-log-csv-tail.png

The screenshots show the live validation results files and a fragment of the structured CSV log. This confirms that results are stored not only in the UI but also in project files.

```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ ls -l results/live_validation_latest.csv results/live_validation_summary.json
cat results/live_validation_summary.json
-rw-rw-r-- 1 artursacc artursacc 6409 May 19 07:03 results/live_validation_latest.csv
-rw-rw-r-- 1 artursacc artursacc 175 May 19 07:03 results/live_validation_summary.json
{
  "timestamp": "2026-05-19T07:03:23",
  "total": 50,
  "pass": 50,
  "failed": 0,
  "false_positive": 0,
  "false_negative": 0,
  "errors": 0,
  "pass_rate_percent": 100.0
}
```

Figure 18: Generated live validation result files.

```
(artursacc@mrktvr)-[~/Desktop/diploma-http-blocking]
$ tail -n 15 logs/http_guard_events.csv
2026-05-20T18:18:33,192.168.199.222,4906,192.168.199.20,80,GET /search?q=K3CscriptK3Ealert(1)K3C2FscriptK3E HTTP/1.1,GET /search?q=script>alert(1)/script> http/1.1,XSS,script,DROP
2026-05-20T18:18:37,192.168.199.222,5006,192.168.199.20,80,GET /login?u=guestK27K200RK201K3D1-- HTTP/1.1,GET /login?u=guest' or 1=1-- http/1.1,SQLI,' or 1=1,DROP
2026-05-20T18:59:29,127.0.0.1,59854,127.0.0.1,80,GET / HTTP/1.1,GET / http/1.1,BENIGN,,ACCEPT
2026-05-20T18:59:29,127.0.0.1,59868,127.0.0.1,80,GET /test?file=../../../../etc/passwd HTTP/1.1,GET /test?file=../../../../etc/passwd http/1.1,TRAVERSAL,..,DROP
2026-05-20T18:59:31,127.0.0.1,59880,127.0.0.1,80,GET /search?q=K3CscriptK3Ealert(1)K3C2FscriptK3E HTTP/1.1,GET /search?q=script>alert(1)/script> http/1.1,XSS,script,DROP
2026-05-20T18:59:33,127.0.0.1,59892,127.0.0.1,80,GET /login?u=guestK27K200RK201K3D1-- HTTP/1.1,GET /login?u=guest' or 1=1-- http/1.1,SQLI,' or 1=1,DROP
2026-05-20T18:59:44,127.0.0.1,43444,127.0.0.1,80,GET / HTTP/1.1,GET / http/1.1,BENIGN,,ACCEPT
2026-05-20T18:59:44,127.0.0.1,43450,127.0.0.1,80,GET / HTTP/1.1,GET / http/1.1,BENIGN,,ACCEPT
2026-05-20T18:59:44,127.0.0.1,43466,127.0.0.1,80,GET /test?file=../../../../etc/passwd HTTP/1.1,GET /test?file=../../../../etc/passwd http/1.1,TRAVERSAL,..,DROP
2026-05-20T18:59:45,127.0.0.1,43476,127.0.0.1,80,GET /search?q=K3CscriptK3Ealert(1)K3C2FscriptK3E HTTP/1.1,GET /search?q=script>alert(1)/script> http/1.1,XSS,script,DROP
2026-05-20T18:59:45,127.0.0.1,43488,127.0.0.1,80,GET /login?u=adminK27K200RK201K3D1-- HTTP/1.1,GET /login?u=admin' or 1=1-- http/1.1,SQLI,' or 1=1,DROP
2026-05-20T19:48:06,127.0.0.1,43832,127.0.0.1,80,GET / HTTP/1.1,GET / http/1.1,BENIGN,,ACCEPT
2026-05-20T19:48:06,127.0.0.1,43834,127.0.0.1,80,GET /test?file=../../../../etc/passwd HTTP/1.1,GET /test?file=../../../../etc/passwd http/1.1,TRAVERSAL,..,DROP
2026-05-20T19:48:08,127.0.0.1,43848,127.0.0.1,80,GET /search?q=K3CscriptK3Ealert(1)K3C2FscriptK3E HTTP/1.1,GET /search?q=script>alert(1)/script> http/1.1,XSS,script,DROP
2026-05-20T19:48:10,127.0.0.1,43462,127.0.0.1,80,GET /login?u=guestK27K200RK201K3D1-- HTTP/1.1,GET /login?u=guest' or 1=1-- http/1.1,SQLI,' or 1=1,DROP
```

Figure 19: Tail of the structured HTTP Guard CSV event log.

Main HTTP Guard Logic

File: 49-http-guard-core-code.png

The screenshot shows the key logic of the HTTP Guard Agent. In this fragment, the request line is normalized and checked through `detect_pattern`; then the category and action are determined. After that, the event is written to a structured log, and the packet receives the decision `packet.drop()` or `packet.accept()`.

```
1 # Core HTTP Guard decision logic
2
3 raw_request_line = extract_request_line(ip)
4
5 if not raw_request_line:
6     packet.accept()
7     return
8
9 normalized_request_line = normalize_request_line(raw_request_line)
10
11 category, matched_pattern = detect_pattern(
12     raw_request_line,
13     normalized_request_line,
14     RULES
15 )
16
17 if category == "BENIGN":
18     action = "ACCEPT"
19 else:
20     action = "DROP"
21
22 write_log(
23     src_ip=ip.src,
24     src_port=tcp.sport,
25     dst_ip=ip.dst,
26     dst_port=tcp.dport,
27     raw_request_line=raw_request_line,
28     normalized_request_line=normalized_request_line,
29     category=category,
30     matched_pattern=matched_pattern,
31     action=action
32 )
33
34 if action == "DROP":
35     packet.drop()
36 else:
37     packet.accept()
```

Figure 20: Core HTTP Guard logic for normalization, detection, logging, and packet decisions.

Appendix C

Source Code Fragments

This appendix lists the main source code and configuration files of the practical implementation.

The full source code of the practical part is located in the project folder.

File	Purpose
proto/nfqueue_http_guard.py	Main HTTP Guard Agent
agent_api/agent_api_server.py	Local Agent API
dashboard/index.html	SIEM-like dashboard
dashboard/generate_dashboard.py	Helper file for the dashboard
config/rules.json	Detection rules
config/endpoints.json	endpoint configuration
config/validation_checks.json	Set of quick validation checks
tests/run_50_tests.py	Script for the controlled experiment
tests/test_cases_50.csv	Dataset of 50 HTTP test cases

Table 1: Main source code and configuration files

The full project is also packaged in the archive:

Diploma_HTTP_Guard_Final_Practical_Part.zip

Appendix D

Experiment Results

This appendix describes the main result files.

File	Purpose
results/final_experiment_results.csv	Per-case results of the controlled experiment on 50 test cases
results/final_experiment_summary.json	Summary statistics of the controlled experiment
results/final_experiment_report.md	Text report for the controlled experiment
results/live_validation_latest.csv	Latest results of full live validation
results/live_validation_summary.json	Summary statistics of live validation
logs/http_guard_events.csv	Runtime event log of HTTP Guard

Table 2: Main experiment result files

The key final indicators are:

Metric	Value
Total test cases	50
Passed	50
Pass rate	100%
Final ACCEPT	20
Final DROP	30
False positives	0
False negatives	0

Table 3: Key final experiment indicators